Arbitrary-Resolution Hyperspectral Pansharpening Neural Operators

Lin He, *Member, IEEE*, Zhou Fang, Jun Li, *Fellow, IEEE*, Hanghui Ye, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Arbitrary-resolution hyperspectral (ARHS) pansharpening seeks to enhance low-resolution hyperspectral (LRHS) images to any target spatial resolutions by collaborating with the connected panchromatic (PAN) images. The ARHS pansharpening task is significantly different from standard hyperspectral (HS) pansharpening, where ARHS pansharpening has to deal with the challenge that how to pansharpen upcoming LRHS images to any desired scales far beyond the training scales. Neural operators (NOs) form mapping between functional spaces and build continuous function relations, offering an alternative potential path to deal with ARHS pansharpening task. In this article, we are dedicated to the design of pansharpening NOs (PNOs) by the postulation that LRHS and PAN are observations of the input functional space while target high-resolution HS (HRHS) images are observed from the output functional space. Our PNOs are built with a paradigm comprising prepansharpening encoding, inverse discretization, and functional mapping subnetwork. More specifically, by use of kernel projection of frequency domain-related design and Galerkin design, we develop Fourier PNO (Fourier-PNO) and Galerkin PNO (Galerkin-PNO). In addition, we propose the concept of dynamic spectral–spatial Green's function and accordingly develop an enhanced Fourier-PNO (eFourier-PNO), and further introduce a PAN-guided test space tactic to develop an enhanced Galerkin-PNO (eGalerkin-PNO). Finally, we discuss and explain some details such as the scale generalization ability of our PNOs. Our four PNOs are tested on three datasets, and the experimental results verify their excellent performances.**

***Index Terms*— Arbitrary-resolution pansharpening, hyperspectral (HS) images, pansharpening neural operators (PNOs).**

Received 25 December 2024; revised 25 February 2025; accepted 19 March 2025. Date of current version 21 April 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62071184 and Grant 42030111 and in part by Guangdong Basic and Applied Basic Research Foundation under Grant 2022A1515011615 and Grant 2023A1515011887. *(Corresponding authors: Lin He; Jun Li.)*

Lin He, Zhou Fang, and Hanghui Ye are with the School of Automation Science and Engineering and Guangdong Engineering Technology Research Center of Unmanned Aerial Vehicle System, South China University of Technology, Guangzhou 510640, China (e-mail: helin@scut.edu.cn).

Jun Li is with the School of Computer Science and the Hubei Key Laboratory of Intelligent Geo-Information Processing, China University of Geosciences, Wuhan 430078, China (e-mail: lijuncug@cug.edu.cn).

Antonio Plaza is with the Hyperspectral Computing Laboratory, University of Extremadura, 10003 Cáceres, Spain (e-mail: aplaza@unex.es).

Digital Object Identifier 10.1109/TGRS.2025.3558086

NOMENCLATURE

$\mathbf{H}$	Observed LRHS image.
$\mathbf{P}$	Observed PAN image.
$\mathbf{H}^{\uparrow}$	Preinterpolated version of $\mathbf{H}$.
$\mathbf{H}_t$	Observed LRHS image for training phase.
$\mathbf{P}_t$	Observed PAN image for training phase.
$\widetilde{\mathbf{H}}_t$	Synthetic reduced-resolution LRHS image for training phase.
$\widetilde{\mathbf{P}}_t$	Synthetic reduced-resolution PAN image for training phase.
$\widetilde{\mathbf{H}}_t^{\uparrow}$	Preinterpolated version of $\widetilde{\mathbf{H}}_t$.
$\widehat{\mathbf{H}}_s$	Target HS image.
s_r	Prepansharpening encoding scale (i.e., spatial resolution ratio between observed PAN and LRHS).
s	ARHS pansharpening scale (i.e., spatial resolution ratio between the ARHS pansharpened image and the LRHS image $\mathbf{H}$).
$\boldsymbol{s}_t$	Training scales (i.e., spatial resolution ratio between $\mathbf{H}$ and $\widetilde{\mathbf{H}}_t$) set for training phase.

I. INTRODUCTION

HYPERSPECTRAL (HS) images accommodate observations with many continuously neighboring spectral bands [1], [2], [3], and nowadays have being successfully applied in many practical fields [3], [4], [5], [6]. However, their spectral resolutions and spatial resolutions may constrain with each other due to the connected imaging mechanism. The obstacle hinders HS data from resolving observed scene in high resolution. To relieve the problem, two solutions could be taken into consideration. One is to develop higher precision optical instruments, but which usually means sharply increasing cost. The other is to resort to imaging computation which enhances data with specifically designed computation. HS pansharpening is one of imaging computation techniques, which uses panchromatic (PAN) images with higher spatial resolutions to collaborate low-resolution HS (LRHS) data to produce high-resolution HS(HRHS) images [2], [7], [8], [9].

HS pansharpening is in fact evolved from traditional multispectral pansharpening [2], [8], [9]. The technique can be grouped into traditional model-driven and modern deep learning. Model-driven HS pansharpening approximately includes hand-crafted approaches and learning-based approaches. Hand-crafted model-driven methods include classical component substitution (CS) and multiresolution analysis

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

(MRA), which use fixed model to stipulate the replacement of HS constituents by associated PAN constituents [10], [11], [12], [13]. While these methods are computationally efficient, they are often considered too simplistic and fail to capture finer details during pansharpening. In contrast, learning-based model-driven methods usually rely on optimization process to tune the model to deal with sophisticated factors among HS and PAN data, instantiations of which include Bayesian estimation, matrix factorization, and variational optimization [14], [15], [16], [17], [18], [19].

In the past decade, HS pansharpening technique has rapidly embraced deep learning, which has advantages of automatic learning of hierarchical representations [20], [21], [22], [23], [24], [25], [26], [27], [28]. Masi et al. [20] innovatively pioneered the convolutional neural network (CNN) thread for multispectral pansharpening, where a three-layer pansharpening CNN (PNN) was designed. Afterward, He et al. [21] developed a novel HS pansharpening specific CNN (HyperPNN) by strengthening the deep spectral representation, and then designed a spectral-fidelity CNN based on the decomposability of HS details [22]. Xie et al. [23] enhanced the reconstruction by redefining the spectral response function in line with HS imaging. Zheng et al. [24] combined channel-spatial attention with residual learning to better reconstruction. Bandara et al. [25] added spatial constraints to the spectral response function for enhancement. Dong et al. [26] designed a feature pyramid scheme to extract multiresolution representations of PAN and HS branches. Bandara and Patel [27] adopted transformer to capture long-range dependencies and better handle complex LRHS and PAN feature interactions. Qu et al. [28] built a iterative optimization-based deep unfolding pansharpening network (PanNet) which enhanced the network interpretability. In addition, deep learning was also applied for tasks related to HS pansharpening, such as HS image super-resolution (HSI-SR) [29], [30], [31], [32], [33], [34]. However, due to being without the cooperation of PAN data, HSI-SR usually yields HRHS reconstruction inferior to HS pansharpening. It is noteworthy that all the above HS pansharpening methods belongs to standard HS pansharpening technique which aims to pansharpen LRHS images to mere the spatial resolutions of the connected PANs.

Recently, arbitrary-resolution HS (ARHS) pansharpening emerges as a promising research field of HS pansharpening, which is significantly distinct from the traditional standard HS pansharpening task. ARHS pansharpening seeks to use PAN images to promote LRHS images to any desired spatial resolutions which may be far beyond that of PANs. As an innovative pansharpening technique, ARHS pansharpening is capable of flexibly adjusting the resolution of pansharpened HS data, and therefore has a great potential in many practical applications [35], such as thematic mapping enhancement [35], [36], [37], [38], [39], object detection [35], [40], [41], and land use classification [35], [38], [42].

Most of existing HS pansharpening methods work in the supervised manner, which means that once the learning under training data is completed in the training phase, the trained pansharpening model is able to pansharpen upcoming data in the succeeding inferring phase [43], [44].

Standard HS pansharpening stipulates that the pansharpening scale (i.e., the spatial ratio of the pansharpened HRHS data and the input LRHS data) keeps invariant across the training phase and the inferring phase, where training process is usually conducted on the reduced-resolution data. That is, the pansharpening scale in the training phase and that in the inferring phase are identical. However, such a scale invariance mechanism across training and inferring does not work anymore in the ARHS pansharpening context since the pansharpening scale can be set to any desired in the inferring phase of ARHS pansharpening. Therefore, a major challenge that ARHS pansharpening has to take on is scale generalization, i.e., how to learn a reliable pansharpening model under fixed pansharpening scales and then generalize it to the inferring phase, which pansharpens future data to HRHS images with any possible desired pansharpening scales [35]. He et al. [35] used a location-sharing mechanism to establish a novel two-stage relay based ARHS pansharpening CNN (ARHS-CNN). Furthermore, they built a novel spatial mask-sharing and generalization mechanism into a variable subpixel convolution-based CNN (VSPC-CNN) to accomplish the ARHS pansharpening task [45].

Over recent years, neural operators (NOs) witness more and more attentions in handling partial differential equations (PDEs) [46], [47], [48], [49], [50], [51], [52], which use neural network to approximate operator mapping between infinite-dimensional functional spaces. Through specific architecture design, they are able to directly resolve PDEs on fine grids after being trained on coarse grids, i.e., they are able to transfer solutions across various resolution grids. Such a discretization invariance could be utilized to improve pansharpening scale generalization of the ARHS pansharpening task. In addition, NOs built on continuous function space are coincide with the continuous characteristics of original spectral–spatial scene, which offer nature representations for ARHS pansharpening. Finally, if built on spectral–spatial joint function, NOs can be easily designed to form spatial super-resolution and spectral super-resolution simultaneously. Compared to implicit neural representations prevailing in arbitrary scale super-resolution usually for compute vision task [53], [54], [55], which use neural networks to represent continuous functions, NOs are directly built on the mapping between functional spaces and hence could unveil more sophisticated relationship between pansharpening input and output. Based on all the above considerations, the NO tactic could offer great potentials for ARHS pansharpening task. In this work, we are dedicated to designing pansharpening NOs (PNOs) for the ARHS pansharpening task. More specifically, contributions of our work can be identified as follows.

1) *Fourier PNO (Fourier-PNO) and Galerkin PNO (Galerkin-PNO):* We form continuous spectral–spatial functional spaces and accordingly build a PNO paradigm comprising prepansharpening encoding, inverse discretization, and functional mapping subnetwork, where the operator-connected inner product is lifted to high-dimensional space and then fulfilled by use of kernel projection of frequency domain-related design

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

and Galerkin design. Accordingly, Fourier-PNO and Galerkin-PNO pansharpening methods are developed.

2) *Dynamic Spectral–Spatial Green's Function and Enhanced Fourier PNO (eFourier-PNO):* To follow the spatial variation of various observed HS scenes, we design a dynamic network process to stimulate a dynamic spectral–spatial Green's function associated with the pansharpening operator. The dynamic spectral–spatial Green's function is able to adapt to the related intermediate features. By use of the dynamic spectral–spatial Green's function, we design a new pansharpening operator, i.e., eFourier-PNO, which offers more data adaptiveness to improve pansharpening result.
3) *PAN-Guided Test Space and Enhanced Galerkin PNO (eGalerkin-PNO):* In the Galerkin finite elements approximation, the trial space is responsible for resolving the approximation function, while the test space is the function space helping build overall approximation error. Taking into consideration that PAN data contain spatial details complement to HS data, we use PAN connected data to guide the test space. Therefore, we develop a novel PNO, i.e., eGalerkin-PNO, to tune the detail recovery ability of the operator.
4) *Discussion and Explanation About Prepansharpening Encoding, Inverse Discretization and, Especially, the Scale Generalization Ability of Our PNOs Are Given:* We explain the prepansharpening encoding in data interaction, redundancy removal, and latent space sharing. For inverse discretization, we explain its interpolation and position-scale encoding operations. Finally, we discuss the scale generalization ability of our PNOs from three perspectives: representation ability, data discretization difference across training and inferring phases, and spatial variation across training and inferring data.

The rest of this article is organized as follows. Section II describes the overall designs of Fourier-PNO and Galerkin-PNO. In Section III, the details of eFourier-PNO and eGalerkin-PNO are given. Some discussions about our PNO models are presented in Section IV. Corresponding experiments and their results on three datasets are provided in Section V. Finally, we draw conclusions and future lines in Section VI.

II. Fourier and Galerkin PNOs

Following the thread that using neural networks to approximate operators across functional spaces, we build continuous spectral–spatial functional spaces and accordingly design two PNOs for the ARHS pansharpening task, i.e., Fourier-PNO and Galerkin-PNO.

A. Problem Consideration

Many scientific problems are inherently governed by implicit physical principles, which can be mathematically represented as operators mapping between functional spaces. Applying such an inspiration to ARHS pansharpening, we have the following pansharpening operator equation:

$$H(x, y, b) = \mathcal{U}_\dagger(L(x, y, b)) \tag{1}$$

where the low-resolution input function $L(x, y, b) \in \mathcal{L}$ and the high-resolution output function $H(x, y, b) \in \mathcal{H}$ are both defined on Hilbert spaces with the bounded domain $D \subset \mathbb{R}^3$, x and y denote the spatial location, b represents the spectral location, and $\mathcal{U}_\dagger$ denotes the pansharpening operator defined as the functional mapping from the space $\mathcal{L}$ to the space $\mathcal{H}$. Obviously, low-spatial resolution HS data and low-spectral resolution PAN data are sampled from the input function $L(x, y, b)$ and HRHS data are observed from the output function $H(x, y, b)$. If the pansharpening operator $\mathcal{U}_\dagger$ is resolved, we would obtain ARHS pansharpened data for the given LRHS and PAN.

In the context of neural network learning for ARHS pansharpening, we can set a neural network $\mathcal{U}(\cdot; \boldsymbol{\theta})$ parameterized by $\boldsymbol{\theta}$ to approximate $\mathcal{U}_\dagger$. In such a situation, the approximation process can be modeled as a risk minimization problem, which corresponds to minimizing the risk function

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{L\sim\mu} \|\mathcal{U}_\dagger(L) - \mathcal{U}(L; \boldsymbol{\theta})\|_{\mathcal{U}}. \tag{2}$$

Obviously, $\mathcal{U}(\cdot; \boldsymbol{\theta})$ is a $\boldsymbol{\theta}$-parameterized NO to implicitly represent the ideal operator $\mathcal{U}_\dagger$.

Suppose $G((x, y, b), (m, n, l))$ be the Green's function of the inverse operator $\mathcal{U}_\dagger^{-1}$ [56], [57], [58], i.e.,

$$\mathcal{U}_\dagger^{-1} G((x, y, b), (m, n, l)) = \delta((x, y, b) - (m, n, l)) \tag{3}$$

where $(m, n, l) \in D$ and $\delta(\cdot)$ denotes the impulse function in the spectral–spatial domain. $L(x, y, b)$ can be represented with the impulse as

$$L(x, y, b) = \int_D L(m, n, l)\delta((x, y, b) - (m, n, l))\mathrm{d}m\mathrm{d}n\mathrm{d}l. \tag{4}$$

Then, to perform the mapping from $L(x, y, b)$ to $H(x, y, b)$, we can alternatively have the following formulation based on the spectral–spatial Green's function $G((x, y, b), (m, n, l))$:

$$\begin{aligned} H(x, y, b) &= \mathcal{U}_\dagger \int_D L(m, n, l)\delta((x, y, b) - (m, n, l))\mathrm{d}m\mathrm{d}n\mathrm{d}l \\ &= \int_D L(m, n, l)\mathcal{U}_\dagger\delta((x, y, b) - (m, n, l))\mathrm{d}m\mathrm{d}n\mathrm{d}l \\ &= \int_D L(m, n, l)G((x, y, b), (m, n, l))\mathrm{d}m\mathrm{d}n\mathrm{d}l \\ &= \langle L(m, n, l), G^*((x, y, b), (m, n, l))\rangle_{m,n,l} \end{aligned} \tag{5}$$

where G^* represents the conjugate of G, and $\langle\cdot, \cdot\rangle_{m,n,l}$ stands for the projection via an inner product that involves an integral over the m, n, l-dimension. If we merely take into consideration ARHS pansharpening along spatial dimension and assume the spectral–spatial Green's function to be real, then (5) is degenerated to

$$\begin{aligned} H(x, y, b) &= \int_{D'} G((x, y, b), (m, n, b))L(m, n, b)\mathrm{d}m\mathrm{d}n \\ &= \langle G((x, y, b), (m, n, b)), L(m, n, b)\rangle_{m,n} \end{aligned} \tag{6}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

where $D' \subset \mathbb{R}^2$ denotes the degenerated domain. Let a fixed but unknown neural network $\kappa((x, y, b), (m, n, l))$ approximate the spectral–spatial Green's function $G((x, y, b), (m, n, l))$, then we have

$$H(x, y, b) = \int_{D'} \kappa((x, y, b), (m, n, b))L(m, n, b)\mathrm{d}m\mathrm{d}n = \langle \kappa((x, y, b), (m, n, b)), L(m, n, b)\rangle_{m,n}. \quad (7)$$

That is, the relationship between the low-resolution data L and the high-resolution data H is hidden in an kernel projection where the kernel $\kappa((x, y, b), (m, n, l))$ is unknown.

B. PNO Scheme

Fig. 1 graphically illustrates the scheme of our PNO design, which consists of three modules including prepansharpening encoding, inverse discretization, and functional mapping subnetwork.

Our PNOs introduce a prepansharpening encoding Ψ in their structures. The achievements of the prepansharpening encoding lie in three folds. First, it builds the interaction of dual-source data PAN and LRHS as a preliminary processing. Second, the encoding strengthens the useful spectral–spatial information by compressing the input data and removing redundancy. Third, the encoding tends to learn the spectral–spatial latent space specific to the input data. More details of its implementation can be found in later Section IV-A.

In practical learning for mapping between functional spaces, input-output sample pairs are needed, i.e., the input and the output of the learning are in the same discretization level. However, in our ARHS pansharpening task, the resolution of the input data and that of the pansharpened HS data are inconsistent. Therefore, we design an inverse discretization process $\mathcal{T}^{-1}(\cdot; s')$ to adjust the discretization level of its input to the required fine grids, so that the learning of functional spaces mapping can be further run in the subsequent process. Besides, a position-scale encoding operation connected to the scale $s' = (s/s_r)$, which records the spatial relation between the pansharpened HS data and the latent code, is included in inverse discretization. The details of the design of inverse discretization can be found in later Section IV-A.

Involving the above prepansharpening encoding and inverse discretization, (7) can be reformulated as

$$H(x, y, b) = \int_{D'} \kappa((x, y, b), (m, n, b))\mathcal{T}^{-1}\Psi L(m, n, b)\mathrm{d}m\mathrm{d}n = \langle \kappa((x, y, b), (m, n, b)), \mathcal{T}^{-1}\Psi L(m, n, b)\rangle_{m,n}. \quad (8)$$

If we tend to iteratively resolve the kernel in the supervised-learning context, and then map the mathematical representation in (6) to a neural network in an unfolding manner, we can have

$$\begin{aligned} H_1(x, y, b) &= \langle \kappa_i((x, y, b), (m, n, b)), \mathcal{T}^{-1}\Psi L(m, n, b)\rangle_{m,n} \\ H_2(x, y, b) &= \langle \kappa_i((x, y, b), (m, n, b)), H_1(m, n, b)\rangle_{m,n} \\ &\vdots \\ H_{i+1}(x, y, b) &= \langle \kappa_i((x, y, b), (m, n, b)), H_i(m, n, b)\rangle_{m,n} \end{aligned} \quad (9)$$

where $i = 0, \ldots, I-1$ denotes the iteration index, and $\kappa_i(x, y, b)$ denotes the kernel connected to the ith unfolding layer. To further improve the resolving ability of the network, the iteration above can be mapped to a higher dimensional operation of the form

$$\begin{aligned} H_0(x, y, b) &= \mathcal{P}(\mathcal{T}^{-1}\Psi L(x, y, b)) \\ H_{i+1}(x, y, b) &= \sigma(\langle \kappa_i((x, y, b), (m, n, b)), H_i(m, n, b)\rangle_{m,n}) \\ \widehat{H}(x, y, b) &= \mathcal{Q}(H_I(x, y, b)) \end{aligned} \quad (10)$$

where the point-wise operation $\mathcal{P}$ lifts the input function to the first hidden representation $L(D; \mathbb{R}^{d_L}) \to H_0(D; \mathbb{R}^{d_{H_0}})$, the point-wise operation $\mathcal{Q}$ projects the last hidden representation to the output function $H_I(D; \mathbb{R}^{d_{H_I}}) \to \widehat{H}(D; \mathbb{R}^{d_H})$, and the activation operator σ is point-wise and enables NOs to model nonlinear components. Equation (10) is connected to the functional mapping subnetwork $\mathcal{G}$, as shown in Fig. 1.

In practical supervised-learning context, a PNO following our scheme (as shown in Fig. 1) works in two phases. In the training phase, the parameterized PNO is trained under the following minimization of the empirical risk instantiated from (2):

$$J\left(\mathbf{H}_t; \mathcal{U}\left(\mathbf{L}_t; s_t', \boldsymbol{\theta}\right)\right) = \sum_{i=1}\sum_{j=1}\left\|\mathbf{H}_t^{(i,j)} - \mathcal{U}\left(\mathbf{L}_t^{(i,\cdot)}; s_t', \boldsymbol{\theta}\right)\right\|_{\mathcal{H}} \quad (11)$$

where s_t' denotes the scaling factor of the inverse discretization in training phase, and (i, j) represents the jth location of the ith patch, which is cropped from the observed data for data augmentation. More specifically, the above empirical risk can be rewritten as

$$J\left(\mathbf{H}_t; \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi(\mathbf{L}_t; \boldsymbol{\theta}_1); s_t'\right); \boldsymbol{\theta}_2\right)\right) = \sum_{i=1}\sum_{j=1}\left\|\mathbf{H}_t^{(i,j)} - \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi\left(\mathbf{L}_t^{(i,\cdot)}; \boldsymbol{\theta}_1\right); s_t'\right); \boldsymbol{\theta}_2\right)\right\|_{\mathcal{H}} \quad (12)$$

where $\boldsymbol{\theta}_1$ denotes the parameters of the prepansharpening encoding, and $\boldsymbol{\theta}_2$ represents the parameters of the functional mapping subnetwork. Referring to Fig. 1, the training procedure can be mathematically represented as

$$\boldsymbol{\theta}_1^{\mathrm{opt}}, \boldsymbol{\theta}_2^{\mathrm{opt}} = \arg\min_{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2} J\left(\mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi(\mathbf{L}_t; \boldsymbol{\theta}_1); s_t'\right); \boldsymbol{\theta}_2\right), \mathbf{H}_t\right). \quad (13)$$

After training, we have the learned model $\mathcal{U}(\cdot; \boldsymbol{\theta}_1^{\mathrm{opt}}, \boldsymbol{\theta}_2^{\mathrm{opt}})$. In the succeeding inferring phase, $\mathcal{U}(\cdot; \boldsymbol{\theta}_1^{\mathrm{opt}}, \boldsymbol{\theta}_2^{\mathrm{opt}})$ is used to infer the ARHS pansharpening result as follows:

$$\widehat{\mathbf{H}}_s = \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi\left(\mathbf{L}; \boldsymbol{\theta}_1^{\mathrm{opt}}\right); s'\right); \boldsymbol{\theta}_2^{\mathrm{opt}}\right) \quad (14)$$

where $\mathbf{L}$ is the concatenation of $\mathbf{H}^{\uparrow}$ and $\mathbf{P}$. s' represents the scaling factor of inverse discretization in the inferring phase.

C. Kernel Projection Calculations

In the above design, the kernel projection in (10) still needs to be resolved while the kernel is unknown. In this section, we present a Fourier-PNO and a Galerkin-PNO based on Fourier-type kernel integral and Galerkin-type kernel integral, which leverage the time-frequency convolution theorem and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

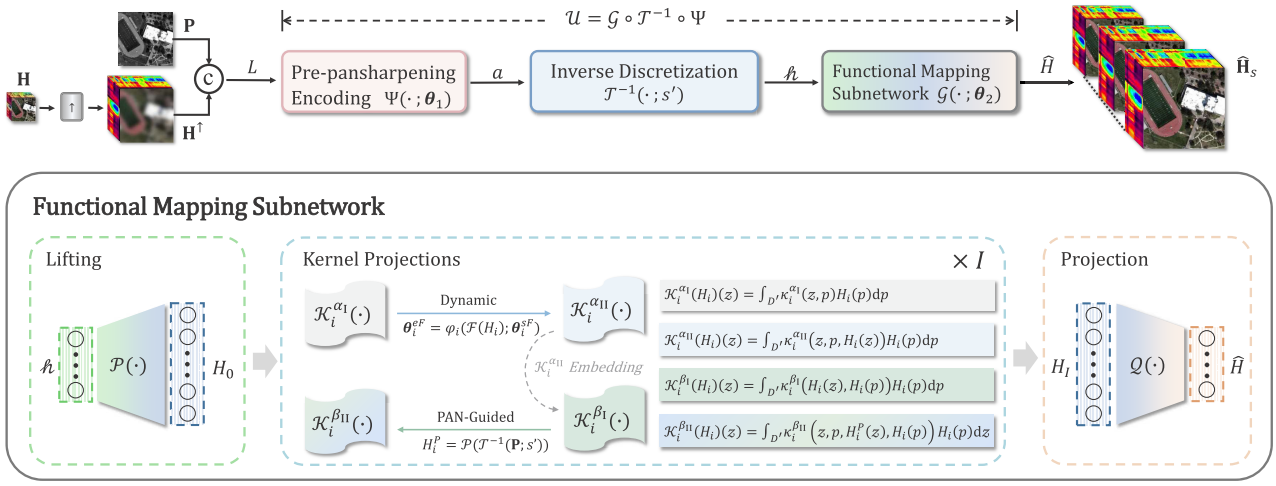


Fig. 1. Overall framework of our PNOs for ARHS pansharpening. ↑ and © stand for the interpolation operation and the concatenation operation, respectively.

Algorithm 1 Fourier-PNO

Input:
LR training data $\mathbf{L}_t$, HR training data $\mathbf{H}_t$, training scales $\boldsymbol{s}_t$; LR inferring data $\mathbf{L}$, inferring scale s;
Output:
Reconstructed HS data $\widehat{\mathbf{H}}_s$ with scale s;

Randomly initialize model parameters $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_F$;
for each epoch **do**
  Obtain latent code a by prepansharpening encoding;
  Obtain $\hbar$ by inverse discretization;
  Obtain H_0 by lifting $\mathcal{P}$;
  for the ith fold **do**
    Calculate $\mathcal{F}(H_i)$ by Fourier transform;
    Calculate $\boldsymbol{\theta}_i^F \cdot \mathcal{F}(H_i)$ by Hadamard product;
    Calculate H_{i+1} using Eq. (17);
  end for
  Obtain $\widehat{H}$ by projection $\mathcal{Q}$;
  Calculate loss function J using Eq. (11);
  Update parameters $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_F$ using Eq. (18);
end for
Obtain the optimal parameters $\boldsymbol{\theta}_1^{opt}$, $\boldsymbol{\theta}_2^{\prime opt}$, and $\boldsymbol{\theta}_F^{opt}$;
return $\widehat{\mathbf{H}}_s$ using Eq. (19);

the Petrov–Galerkin projection of finite elements to efficiently perform integral computations, respectively. For brevity, hereafter, we denote (x, y, b), (m, n, b), and $\mathrm{d}m\mathrm{d}n$ as z, p, and $\mathrm{d}p$, respectively. Then, the Fourier-type integral is associated with the following kernel projection:

$$\begin{aligned}\mathcal{K}_i^{\alpha_1}(H_i)(z) &= \left\langle \kappa_i^{\alpha_1}(z,p), H_i(p)\right\rangle \\ &= \int_{D'} \kappa_i^{\alpha_1}(z,p)H_i(p)\mathrm{d}p \end{aligned} \tag{15}$$

where $\mathcal{K}_i^{\alpha_1}: D'\times D' \to \mathbb{R}^{d_{H_{i+1}}\times d_{H_i}}$ represents the Fourier-type kernel integral in the ith hidden layer. Considering the supervised-learning setting, the above kernel projection can be relaxed as the Galerkin-type integral of the form

$$\begin{aligned}\mathcal{K}_i^{\beta_1}(H_i)(z) &= \left\langle \kappa_i^{\beta_1}(H_i(z),H_i(p)), H_i(p)\right\rangle \\ &= \int_{D'} \kappa_i^{\beta_1}(H_i(z),H_i(p))H_i(p)\mathrm{d}p \end{aligned} \tag{16}$$

where $\mathcal{K}_i^{\beta_1}: \mathbb{R}^{d_{H_i}+d_{H_i}} \to \mathbb{R}^{d_{H_{i+1}}\times d_{H_i}}$ denotes the Galerkin-type kernel integral in the ith hidden layer. Their specific computational processes are in the following.

1) Fourier Kernel Integral: Fourier transform reveals the frequency characteristic of a signal [49], [59]. In (15), letting $\kappa_i^{\alpha_1}(z,p) = \kappa_i^{\alpha_1}(z-p)$ and then applying the convolution theorem, the Fourier integral can be transformed to frequency Hadamard product as shown in Fig. 2(a), thereby reducing the computational complexity of kernel integral to nearly linear, which can be of the form

$$\begin{aligned}\mathcal{K}_i^{\alpha_1}(H_i)(z) &= \kappa_i^{\alpha_1}(z) * H_i(z) \\ &= \mathcal{F}^{-1}\left(\mathcal{F}\left(\kappa_i^{\alpha_1}\right)\cdot\mathcal{F}(H_i)\right)(z) \\ &= \mathcal{F}^{-1}\left(\boldsymbol{\theta}_i^F\cdot\mathcal{F}(H_i)\right)(z)\end{aligned} \tag{17}$$

where $\mathcal{F}$ and $\mathcal{F}^{-1}$ stand for the Fourier transform and its inverse, respectively, and $\boldsymbol{\theta}_i^F$ is the result of the Fourier transform on a periodic function $\kappa_i^{\alpha_1}$. In this way, the Green's function κ^{α_1} can be learned in the frequency domain since the Fourier integral can capture frequency characteristics of the input data and simulate these characteristics through the neural network. The structure of Fourier kernel integral can be found in Fig. 2(a).

From above consideration, the training process of the Fourier PNO is to resolve

$$\begin{aligned}&\boldsymbol{\theta}_1^{\mathrm{opt}}, \boldsymbol{\theta}_2^{\prime\mathrm{opt}}, \boldsymbol{\theta}_F^{\mathrm{opt}} \\ &= \arg\min_{\boldsymbol{\theta}_1,\boldsymbol{\theta}_2',\boldsymbol{\theta}_F} J\left(\mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi(\mathbf{L}_t;\boldsymbol{\theta}_1);s_t'\right);\boldsymbol{\theta}_2',\mathcal{K}^{\alpha_1}(\cdot;\boldsymbol{\theta}_F)\right),\mathbf{H}_t\right)\end{aligned} \tag{18}$$

where $\boldsymbol{\theta}_F$ and $\boldsymbol{\theta}_2'$ stand for the kernel parameters and other parameters of $\boldsymbol{\theta}_2$ in Fourier kernel integral, respectively. And the connected inferring phase is to compute

$$\widehat{\mathbf{H}}_s = \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi\left(\mathbf{L};\boldsymbol{\theta}_1^{\mathrm{opt}}\right);s'\right);\boldsymbol{\theta}_2^{\prime\mathrm{opt}},\mathcal{K}^{\alpha_1}\left(\cdot;\boldsymbol{\theta}_F^{\mathrm{opt}}\right)\right). \tag{19}$$

Therefore, we have a Fourier PNO as in Algorithm 1.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

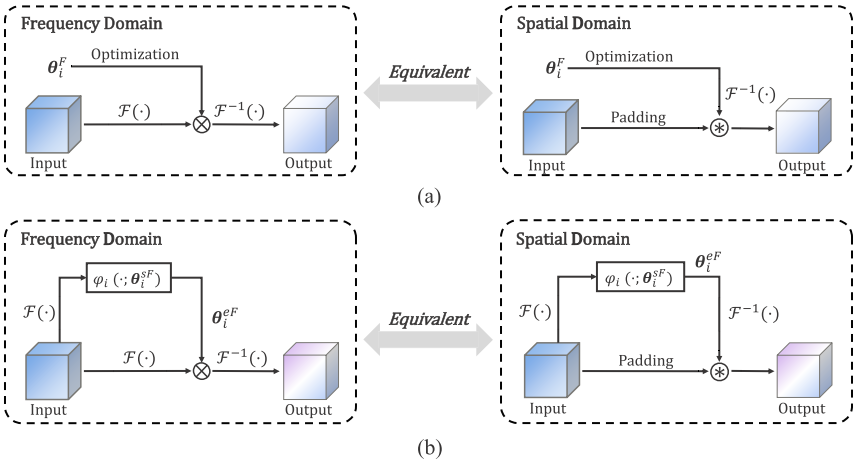


Fig. 2. Kernel projections in two Fourier NOs. (a) In Fourier-PNO and (b) in eFourier-PNO. $\circledast$ and $\otimes$ denote the convolution and the element-wise product, respectively.

Algorithm 2 Galerkin-PNO

Input:
$\mathbf{L}_t$, $\mathbf{H}_t$, $\boldsymbol{s}_t$, $\mathbf{L}$, s;
Output:
$\widehat{\mathbf{H}}_s$;
Randomly initialize $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_G$;
for each epoch **do**
  Obtain a, τ_ω, $\hbar$ and H_0, sequentially;
  for the ith fold **do**
    Obtain q, k, and v by linear transformation;
    Obtain $\bar{k}$ and $\bar{v}$ by layer normalization;
    Obtain H_{i+1} Eq. (20);
  end for
  Obtain $\widehat{H}$;
  Calculate J using Eq. (11);
  Update $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_G$ using Eq. (21);
end for
Obtain $\boldsymbol{\theta}_1^{opt}$, $\boldsymbol{\theta}_2^{\prime opt}$, and $\boldsymbol{\theta}_G^{opt}$;
return $\widehat{\mathbf{H}}_s$ using Eq. (22);

2) Galerkin Kernel Integral: Galerkin kernel integral is a finite element approximation approach a little similar to self-attention [60], [61]. Let $q, k, and v$ denote the query, key, and value functions derived from the feature H_i, respectively, and $n_\diamond$ ($\diamond \in \{L, H\}$) denotes the grid size of $\diamond$. The Galerkin-type attention is given by $\mathcal{K}_i^{\beta_1}(H_i) = (1/n_H)q(\bar{k}^T\bar{v})$, where $\bar{k}$ and $\bar{v}$ denote the layer-normalized features of k and v, respectively. As indicated, the Galerkin-type attention is to some degree different from the original self-attention. It adopts layer-normalization, instead of softmax, to achieve the interpretation of finite element theory-based Petrov–Galerkin projection, and in addition a linear computation structure to achieve low computation complexity [61]. Let j and l denote the column index of the associated feature, and $\kappa_i^{\beta_1}(H_i(z), H_i(p)) = (e^{\langle\bar{k}(z),\bar{v}(p)\rangle}/\int_{D'} e^{\langle\bar{k}(z),\bar{v}(p)\rangle}\mathrm{d}p)W_{qi}$ in (16) represents an exponent-based kernel, where $W_{qi} \in \mathbb{R}^{d_{H_{i+1}} \times d_{H_i}}$ denotes the transformation of q in the ith fold, and hereinafter $\langle\cdot,\cdot\rangle$ specifically represents the dot product. Then, in the discrete setting, the Galerkin kernel integral can be reformulated as

$$\left(\mathcal{K}_i^{\beta_1}(H_i)(z)\right)_j = \frac{1}{n_H}\sum_{l=1}^{d_{H_{i+1}}}\langle\bar{k}_l, \bar{v}_j\rangle q_l(z) \approx \sum_{l=1}^{d_{H_{i+1}}}\left(\int_{D'}\bar{k}_l(p)\bar{v}_j(p)\mathrm{d}p\right)q_l(z). \tag{20}$$

Note that $\sum_{l=1}^{d_{H_{i+1}}}(\int_{D'}k_l(p)v_j(p)\mathrm{d}p)q_l(z)$ represents a Petrov–Galerkin projection [62], where $\{q_l(\cdot)\}_{l=1}^{d_{H_{i+1}}}$ and $\{k_l(\cdot)\}_{l=1}^{d_{H_{i+1}}}$ can be viewed as bases in the trial function space and the test function space [62], respectively. Consequently, Galerkin-type attention mechanism performs the kernel integral by a finite element approximation in function space. The structure of Galerkin kernel integral can be found in Fig. 3(a).

Therefore, we have the training process associated with the Galerkin PNO as

$$\boldsymbol{\theta}_1^{\mathrm{opt}}, \boldsymbol{\theta}_2^{\prime\mathrm{opt}}, \boldsymbol{\theta}_G^{\mathrm{opt}} = \arg\min_{\boldsymbol{\theta}_1,\boldsymbol{\theta}_2',\boldsymbol{\theta}_G} J\left(\mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi(\mathbf{L}_t;\boldsymbol{\theta}_1);s_t'\right);\boldsymbol{\theta}_2', \mathcal{K}^{\beta_1}(\cdot;\boldsymbol{\theta}_G)\right), \mathbf{H}_t\right) \tag{21}$$

where $\boldsymbol{\theta}_G = \{\boldsymbol{\theta}_i^G\}_{i=0}^{I-1}$ denote the Galerkin kernel integral parameters in $\boldsymbol{\theta}_2$. The corresponding inferring phase is as follows:

$$\widehat{\mathbf{H}}_s = \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi\left(\mathbf{L};\boldsymbol{\theta}_1^{\mathrm{opt}}\right);s'\right);\boldsymbol{\theta}_2^{\prime\mathrm{opt}}, \mathcal{K}^{\beta_1}\left(\cdot;\boldsymbol{\theta}_G^{\mathrm{opt}}\right)\right). \tag{22}$$

Then, we can present a Galerkin PNO as in Algorithm 2.

III. Enhanced Fourier and Galerkin PNOs

The above Fourier-PNO and Galerkin-PNO do not explicitly take into consideration possible spatial variations in the observed scenes, which means the spectral characteristics of HS and PAN scenes may vary spatially. By following the spatial variation, PNOs may have potentials to improve

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

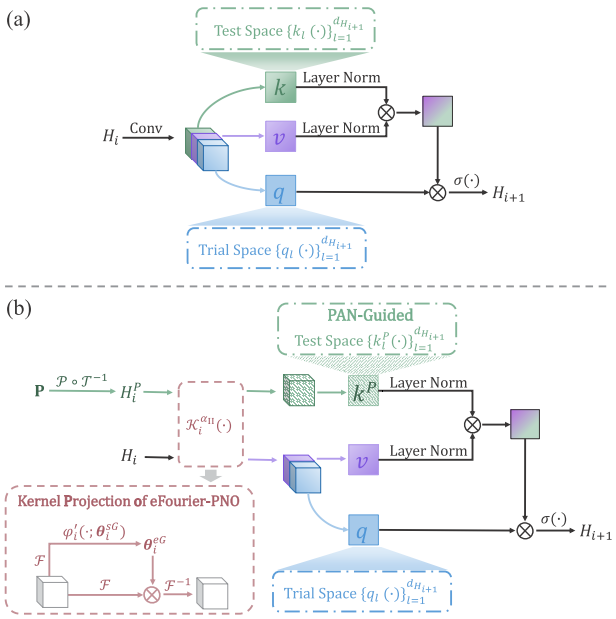


Fig. 3. Kernel projection in two Galerkin NOs. (a) In Galerkin-PNO and (b) in eGalerkin-PNO.

their performance. Moreover, additional considerations such as PAN-guided spatial enhancement could be strengthened.

Therefore, here we propose two enhanced PNOs, i.e., eFourier-PNO and eGalerkin-PNO. The former involves the kernel projection of the form

$$\mathcal{K}_i^{\alpha_{\text{II}}}(H_i)(z) = \left\langle \kappa_i^{\alpha_{\text{II}}}(z, p, H_i(z)), H_i(p) \right\rangle = \int_{D'} \kappa_i^{\alpha_{\text{II}}}(z, p, H_i(z)) H_i(p) \mathrm{d}p \tag{23}$$

where $\mathcal{K}_i^{\alpha_{\text{II}}}: D' \times D' \times \mathbb{R}^{d_{H_i}} \to \mathbb{R}^{d_{H_{i+1}} \times d_{H_i}}$ represents the ith enhanced Fourier kernel integral in hidden layers. The latter is associated with the kernel projection of the form

$$\mathcal{K}_i^{\beta_{\text{II}}}(H_i)(x) = \left\langle \kappa_i^{\alpha_{\text{II}}}(z, p, H_i(z), H_i(p)), H_i(p) \right\rangle = \int_{D'} \kappa_i^{\beta_{\text{II}}}(z, p, H_i^P(z), H_i(p)) H_i(p) \mathrm{d}p \tag{24}$$

where $\mathcal{K}_i^{\beta_{\text{II}}}: D' \times D' \times \mathbb{R}^{d_{H_i}} \times \mathbb{R}^{d_{H_i}} \to \mathbb{R}^{d_{H_{i+1}} \times d_{H_i}}$ stands for the ith enhanced Galerkin kernel integral in hidden layers. The description of PAN deep feature H_i^P is given in Section III-B.

A. Dynamic Spectral–Spatial Green's Function and eFourier-PNO

In the implementation of Fourier-PNO, the spectral–spatial Green's function (i.e., the kernel $\kappa^{\alpha_{\text{I}}}$) is fixed after training and also the kernel parameter $\boldsymbol{\theta}_F$ is with a specific discrete mode (i.e., a fixed kernel size in the frequency domain). However, practical HS scenes may be complex and there exists noneligible spatial variations between the training scenes and the scenes need to pansharpen in the inferring phase.

To tackle the problem, as shown in Fig. 2(b), we design a dynamic spectral–spatial Green's function

$$\boldsymbol{\theta}_i^{eF}(z) = \varphi_i\left(\mathcal{F}(H_i); \boldsymbol{\theta}_i^{sF}\right)(z) \tag{25}$$

where the branch φ_i parameterized by $\boldsymbol{\theta}_i^{sF}$ is to generate the dynamic parameters $\boldsymbol{\theta}_i^{eF}$ adaptive to H_i. For given φ_i, its output $\boldsymbol{\theta}_i^{eF}$ representing the spectral–spatial Green's function varies with the variation of the input H_i. In the training-inferring context, once the branch φ_i is determined by the training, the spectral–spatial Green's function $\boldsymbol{\theta}_i^{eF}$ would dynamically follow the future data in the inferring phase. That is, the dynamic spectral–spatial Green's function is able to adapt to the fluctuation of intermediate feature H_i. Letting

$$\kappa_i^{\alpha_{\text{II}}}(z, p, H_i(z)) = \kappa_i^{\alpha_{\text{II}}}\left(z - p; \boldsymbol{\theta}_i^{eF}(z)\right) \tag{26}$$

then the kernel projection of eFourier-PNO can be mathematically represented as

$$\begin{aligned} \mathcal{K}_i^{\beta_{\text{II}}}(H_i)(z) &= \mathcal{F}^{-1}\left(\varphi_i(\mathcal{F}(H_i)); \boldsymbol{\theta}_i^{sF}\right) * H_i(z) \\ &= \mathcal{F}^{-1}\left(\boldsymbol{\theta}_i^{eF}\right) * H_i(z) \\ &= \mathcal{F}^{-1}\left(\boldsymbol{\theta}_i^{eF} \cdot \mathcal{F}(H_i)\right)(z). \end{aligned} \tag{27}$$

Note that the $\boldsymbol{\theta}_i^{eF}$ is a tensor with the same shape as the input H_i, which means eFourier-PNO owns the global receptive field in the frequency domain, and thus also helps to reconstruct more fine spatial details.

In practical training phase of eFourier-PNO, its parameters can be solved as

$$\begin{aligned} &\boldsymbol{\theta}_1^{\text{opt}}, \boldsymbol{\theta}_2^{\prime\text{opt}}, \boldsymbol{\theta}_{sF}^{\text{opt}} \\ &\quad = \arg\min_{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2', \boldsymbol{\theta}_{sF}} J\Big(\mathcal{G}\big(\mathcal{T}^{-1}\big(\Psi(\mathbf{L}_t; \boldsymbol{\theta}_1); s_t'\big) \\ &\qquad \times \boldsymbol{\theta}_2', \mathcal{K}^{\alpha_{\text{II}}}(\cdot; \boldsymbol{\theta}_{eF} = \varphi(\cdot; \boldsymbol{\theta}_{sF}))\big), \mathbf{H}_t\Big). \end{aligned} \tag{28}$$

Then, the connected inferring phase is to produce target pansharpening result

$$\widehat{\mathbf{H}}_s = \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi\left(\mathbf{L}; \boldsymbol{\theta}_1^{\text{opt}}\right); s'\right); \boldsymbol{\theta}_2^{\prime\text{opt}}, \mathcal{K}^{\alpha_{\text{II}}}\left(\cdot; \boldsymbol{\theta}_{eF} = \varphi\left(\cdot; \boldsymbol{\theta}_{sF}^{\text{opt}}\right)\right)\right). \tag{29}$$

The implementation of our eFourier-PNO can be found in Algorithm 3.

B. PAN-Guided Test Space and eGalerkin-PNO

In the Galerkin method, the trial space is the function space to resolve an approximation function while the test space is used to form the overall approximation error of the estimated function [61].

Here, in addition to the dynamically adaptive ability, we take a further consideration. As PAN data provide more spatial detail than LRHS images, we design a PAN-guided test space to form an eGalerkin-PNO so as to improve spatial detail reconstruction for pansharpening. In such a design, as shown in Fig. 3(b), the test space, jointly with the trial space, helps to minimize the approximation error during the spatial enhancement, and thus resolve high-quality trial space bases for spatial reconstruction. Therefore, eGalerkin-PNO could reconstruct HS data preserving spatial-fidelity. In particular, we first resort to inverse discretization $\mathcal{T}^{-1}(\cdot; s')$ to resize PAN data $\mathbf{P}$ and then lift it to high-dimensional space to obtain deep PAN feature H_i^P

$$H_i^P(z) = \mathcal{P}\left(\mathcal{T}^{-1}\left(\mathbf{P}; s'\right)\right)(z). \tag{30}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 eFourier-PNO

Input:
$\mathbf{L}_t$, $\mathbf{H}_t$, $\boldsymbol{s}_t$, $\mathbf{L}$, s;
Output:
$\widehat{\mathbf{H}}_s$;
Randomly initialize $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_{sF}$;
for each epoch **do**
  Obtain a, τ_ω, $\hbar$ and H_0, sequentially;
  for the ith fold **do**
    Calculate $\mathcal{F}(H_i)$;
    Obtain $\boldsymbol{\theta}_i^{eF}$ using Eq. (25);
    Calculate $\boldsymbol{\theta}_i^{eF} \cdot \mathcal{F}(H_i)$;
    Calculate H_{i+1} using Eq. (27);
  end for
  Obtain $\widehat{H}$;
  Calculate J using Eq. (11);
  Update $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_{sF}$ using Eq. (28);
end for
Obtain $\boldsymbol{\theta}_1^{opt}$, $\boldsymbol{\theta}_2^{\prime opt}$, and $\boldsymbol{\theta}_{sF}^{opt}$;
return $\widehat{\mathbf{H}}_s$ using Eq. (29);

Algorithm 4 eGalerkin-PNO

Input:
$\mathbf{L}_t$, $\mathbf{H}_t$, $\boldsymbol{s}_t$, $\mathbf{L}$, s;
Output:
$\widehat{\mathbf{H}}_s$;
Randomly initialize $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_{sG}$;
for each epoch **do**
  Obtain a, τ_ω, $\hbar$ and H_0, sequentially;
  for the ith fold **do**
    Obtain H_i^P by Eq. (30) on $\mathbf{P}$;
    Obtain q, k^P and v using Eq. (27);
    Obtain $\bar{k}^P$ and $\bar{v}$ by layer normalization;
    Obtain H_{i+1} using Eq. (24);
  end for
  Obtain $\widehat{H}$;
  Calculate J using Eq. (11);
  Update $\boldsymbol{\theta}_1$, $\boldsymbol{\theta}_2'$, and $\boldsymbol{\theta}_{sG}$ using Eq. (32);
end for
Obtain $\boldsymbol{\theta}_1^{opt}$, $\boldsymbol{\theta}_2^{\prime opt}$, and $\boldsymbol{\theta}_{sG}^{opt}$;
return $\widehat{\mathbf{H}}_s$ using Eq. (33);

Then, as shown in Fig. 3(b), we set H_i^P as the original feature of PAN-guided feature k^P, which forms the PAN-guided test space $\{k_l^P(\cdot)\}_{l=1}^{d_{H_{i+1}}}$, and embed $\mathcal{K}^{\alpha_{\text{II}}}$ into $\mathcal{K}^{\beta_{\text{II}}}$ to replace linear transformations of query, key, and value. By the dynamic Green's function, more reliable spatial details are contained in q, k^P and v. Then, the form of the kernel $\kappa_i^{\beta_{\text{II}}}$ in eGalerkin-PNO can be formulated as

$$\kappa_i^{\beta_{\text{II}}}\left(z,\, p,\, H_i^P(z),\, H_i(p)\right) = \frac{e^{\langle \bar{k}^P(z), \bar{v}(p)\rangle}}{\int_{D'} e^{\langle \bar{k}^P(z), \bar{v}(p)\rangle} \mathrm{d}p} \boldsymbol{\theta}_{qi}^{eF} \tag{31}$$

where $\boldsymbol{\theta}_{qi}^{eF} = \varphi_{qi}(\mathcal{F}(H_i))$ stands for the dynamic Green's function in ith fold for the query transformation.

In practical training phase, the optimization of eGalerkin-PNO can be performed as follows:

$$\begin{aligned}\boldsymbol{\theta}_1^{\text{opt}}, \boldsymbol{\theta}_2^{\prime\text{opt}}, \boldsymbol{\theta}_{sG}^{\text{opt}} &= \arg\min_{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2', \boldsymbol{\theta}_{sG}} J\Big(\mathcal{G}\big(\mathcal{T}^{-1}\big(\Psi(\mathbf{L}_t; \boldsymbol{\theta}_1); s_t'\big) \\ &\quad \times \boldsymbol{\theta}_2', \mathcal{K}^{\beta_{\text{II}}}\big(\cdot, \mathbf{P}; \boldsymbol{\theta}_{eG} = \varphi'(\cdot; \boldsymbol{\theta}_{sG})\big)\big)\Big), \mathbf{H}_t\Big)\end{aligned} \tag{32}$$

where $\boldsymbol{\theta}_{eG}$ denotes the dynamic Green's function in $\mathcal{K}^{\beta_{\text{II}}}$, and $\boldsymbol{\theta}_{sG}$ denotes the parameters of the dynamic branches φ' used in $\mathcal{K}^{\beta_{\text{II}}}$. The corresponding inferring phase is of the form

$$\widehat{\mathbf{H}}_s = \mathcal{G}\big(\mathcal{T}^{-1}\big(\Psi\big(\mathbf{L}; \boldsymbol{\theta}_1^{\text{opt}}\big); s'\big); \boldsymbol{\theta}_2^{\prime\text{opt}}, \mathcal{K}^{\beta_{\text{II}}}\big(\cdot, \mathbf{P}; \boldsymbol{\theta}_{eG} = \varphi'\big(\cdot; \boldsymbol{\theta}_{sG}^{\text{opt}}\big)\big)\big). \tag{33}$$

The implementation of our eGalerkin-PNO can be found in Algorithm 4.

IV. More Details

In our PNO design, some details deserve further discussion and explanation.

A. Prepansharpening Encoding and Inverse Discretization

As designed in Section II-B, each of our PNOs comprises a prepansharpening encoding module and an inverse discretization module (as shown in Fig. 1). Prepansharpening encoding is employed to produce a latent space which interacts LRHS and PAN, remove redundancy and capture the inherent information specific to input data. In our design, we employ standard pansharpening CNN Ψ to fulfill such an encoding, yielding latent code with the spatial resolution of PAN but with band number less than the number of HS bands.

As regards the inverse discretization process $\mathcal{T}^{-1}(\cdot; s')$ which inversely discretizes data to perform a rough resolution improvement, it consists of two operations, as shown in Fig. 4. One operation is to interpolate the latent code a to the interpolated feature $a^{\uparrow}$ which is in the same discretization level as the pansharpened HS data. The other operation is a process of position-scale encoding, which records the spatial relation between the pansharpened HS data $\widehat{\mathbf{H}}_s$ and the latent code a. More specifically, we set the zoom scale equal to $s' = (s/s_r)$ in the interpolation operation. This plug-in scheme is presumed not to hurt the representation ability of PNOs, such as discretization invariance [49], [63]. Related qualitative analysis is given in Section IV-B. To further strengthen the inverse discretization, we design a position-scale encoding process, which involves coordinate normalization, offset projection, and scale operation. Absolute coordinates of input and output data are first uniformly normalized in the range (0, 1); and then an offset projection $\tau_\omega(\boldsymbol{c}) = \boldsymbol{c}_\omega - \boldsymbol{c}$ to calculate the final relative coordinate (i.e., coordinate distance $\tau_\omega(\boldsymbol{c})$) between the fine grid $\boldsymbol{c}$ and its neighboring coarse grid $\boldsymbol{c}_\omega$ (as shown in Fig. 4). Besides, ARHS pansharpening scale s is concatenated with τ_ω and $a_\omega^{\uparrow}$ to obtain the hybrid feature

$$\hbar(\boldsymbol{c}) = \left[\left\{a_\omega^{\uparrow}(\boldsymbol{c}), \tau_\omega(\boldsymbol{c})\right\}_{\omega\in\Omega}, s\right] \tag{34}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

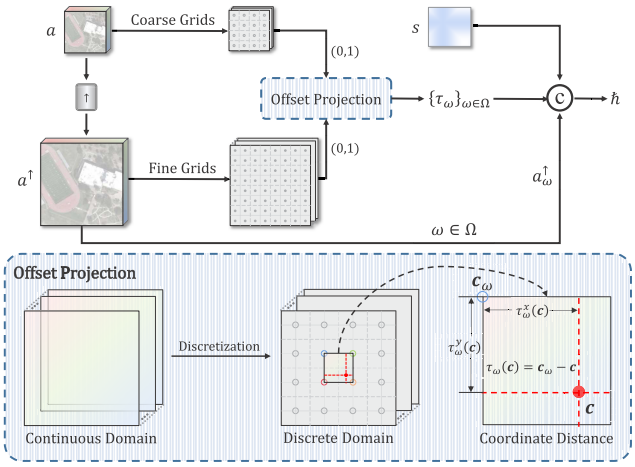


Fig. 4. Block diagram of inverse discretization. Solid dots and hollow dots represent fine grids and coarse grids, respectively. (0, 1) denotes the coordinate normalization operation.

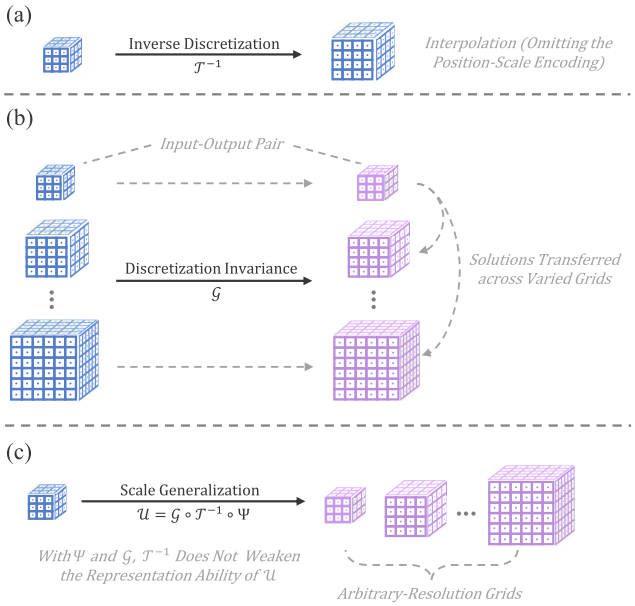


Fig. 5. Block diagram of three discretization situations. (a) Gap between intra-pair grids. (b) Gap between inter-pair grids. (c) Gap between different target resolution grids.

where $[\cdot,\cdot]$ stands for the concatenation operation, and ω denotes the element in the neighboring area Ω.

B. Scale Generalization Ability of Our PNOs

The major challenge faced with ARHS pansharpening task is scale generalization, which means learning a suitable or correct pansharpening model under preset scales in the training phase and then pansharpening future data with any desired scales in the inferring phase, i.e., to generalize the suitable pansharpening model learned under fixed training scales to any pansharpening scales for future data. In this section, we will discuss and explain the scale generalization ability of our PNOs. The generalization ability of PNOs in the training-inferring context is an extremely sophisticated problem, which involves many factors including PNO representation ability, data discretization difference across training and inferring phases, spatial variation across training and inferring data.

The convergence of a NO on discretized grids to the ideal operator can be measured by discretization invariance [49], which means that the NO's representation errors are limited under given discretization levels. Thus, the NO with discretization invariance has strong representation ability. In the following, we qualitatively argue that our PNOs have strong representation abilities. Every PNO of ours consists of a prepansharpening encoding, an inverse discretization, and a functional mapping subnetwork, which could be formulated as (omitting the parameters $\boldsymbol{\theta}$, $\boldsymbol{\theta}_1$, and $\boldsymbol{\theta}_2$)

$$\begin{aligned}\mathcal{U}(\mathbf{L}, s; s') &= \mathcal{G}\left(\mathcal{T}^{-1}\left(\Psi([\mathbf{H}^{\uparrow}, \mathbf{P}]), s; s'\right)\right) \\ &= \mathcal{G}\left(\left[\left\{a_{\omega}^{\uparrow}, \tau_{\omega}\right\}_{\omega\in\Omega}, s\right]\right). \end{aligned} \tag{35}$$

Kovachki et al. [49] pointed out that a point-wise lifting and projection-based NO mapping is discretization invariant, which could be described in Fig. 5(b). It is in accordance with the situation of our functional mapping subnetwork design, and therefore the functional mapping subnetworks in our PNOs have strong representation abilities. Then, we explain that the inverse discretization $\mathcal{T}^{-1}$ shown in Fig. 5(a) does not harm to the representation ability of the connected functional mapping subnetwork $\mathcal{G}$, and therefore the representation ability of each of our PNOs is ensured. First, the prepansharpening encoding Ψ strongly represents the input LRHS and PAN data as features in the latent space, so that neither the preinterpolation of $\mathbf{H}$ to $\mathbf{H}^{\uparrow}$ nor the feature interpolation of a to $a^{\uparrow}$ in the inverse discretization weakens the feature representativeness. Second, the functional mapping subnetwork $\mathcal{G}$ is able to learn sophisticated mapping between the inverse discretization feature and the final pansharpening output to form the target transformation from the input functional space to the output functional space for our ARHS pansharpening task. Therefore, our PNOs have strong representation abilities to achieve the scale generalization, as illustrated in Fig. 5(c).

In ARHS pansharpening, the pansharpening scale in the inferring phase is not preset and could be any desired, which may be significantly different from the training scale. In this situation, the distinction between the discretization level of the target pansharpened data in the training phase and that in the inferring phase is arbitrary. In our PNOs, coordinate normalization is first used to turn coarse grids of LR and fine grids of HR data into the same spatial scope of a uniform coordinate system, and then offset projection is used to obtain the spatial offset τ_ω of fine grids $\boldsymbol{c}$ relative to their neighboring coarse grids $\boldsymbol{c}_\omega$. Therefore, the learned PNO model contains explicit spatial correlations between spatial grids of LR input and HR pansharpened result in our pansharpening task [35]. Moreover, the scale s is encoded into the coordinate processing to form the position-scale encoding which further strengthens the spatial correlations. Therefore, the position correlations between the input data and the output data hidden in the trained PNO model help the model to be generalized to the future inferring data with different pansharpening scales.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PROPERTIES OF DATASETS USED FOR EVALUATION

Datasets	Spectral Range	Original HS/PAN	Training HS/PAN	Inferring HS/PAN	Instrument/Source
Merced	0.4-2.5 μm	2100*180*242/6300*540	600*180*134/1800*540	60*60*134/180*180	Hyperion and ALI
Houston	0.38-1.05 μm	349*1905*144/	117*217*144/351*651	60*60*144/180*180	GRSS
Washington	0.4-0.76 μm	1280*300*210/	138*75*191/552*300	45*45*191/180*180	HYDICE

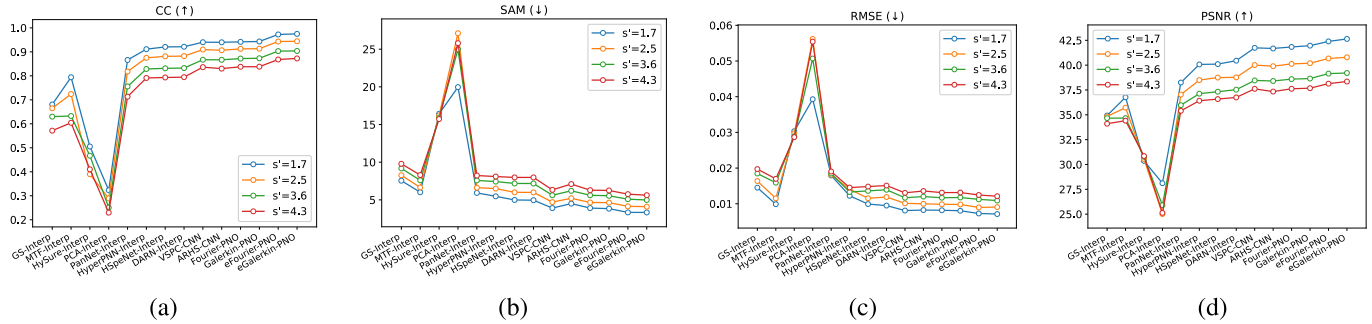


Fig. 6. Pansharpening performances under arbitrary scales s on Merced dataset. (a) CC, (b) SAM, (c) RMSE, and (d) PSNR.

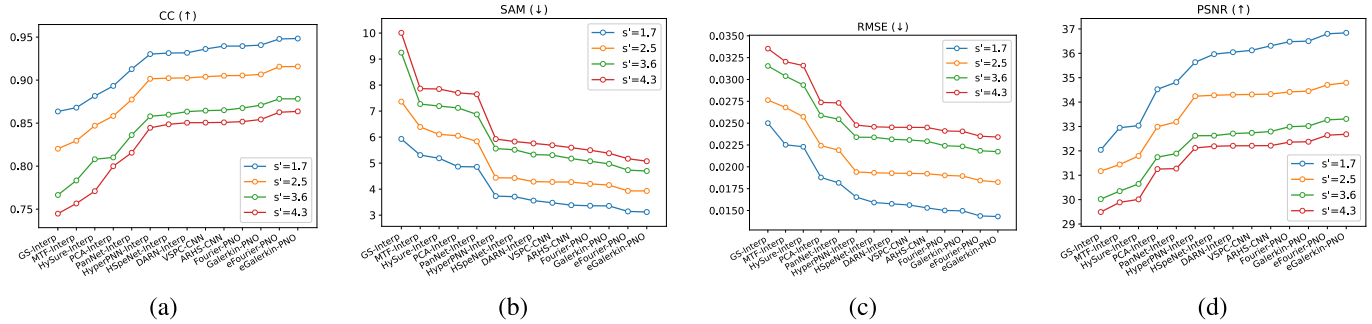


Fig. 7. Pansharpening performances under arbitrary scales s on Houston dataset. (a) CC, (b) SAM, (c) RMSE, and (d) PSNR.

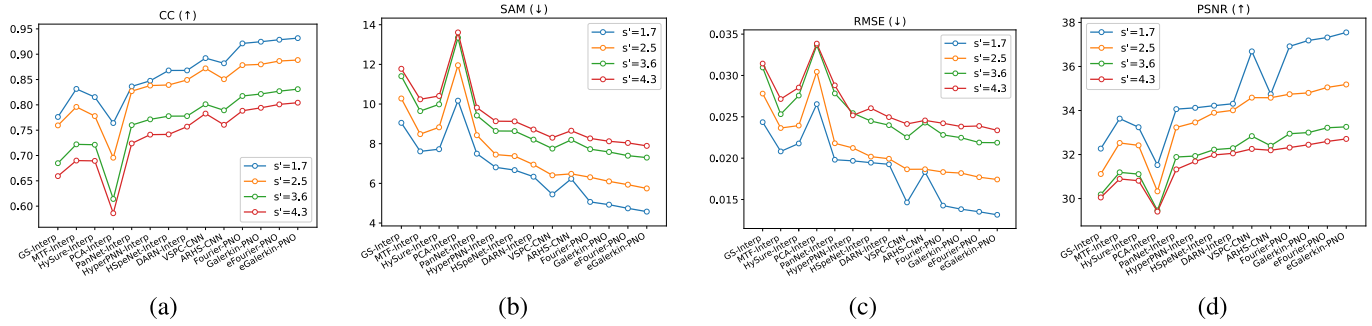


Fig. 8. Pansharpening performances under arbitrary scales s on Washington dataset. (a) CC, (b) SAM, (c) RMSE, and (d) PSNR.

In training-inferring context, the scenes connected to training data and the scenes connected to the future inferring data may significantly different. Thus, the spatial characteristic of training data and that of inferring data are varying. In our eFourier-PNO and eGalerkin-PNO, we design dynamic learn-to-learn branches φ to generate adaptive pansharpening rules, which enable some dynamic parameters $\boldsymbol{\theta}_{eF}$ are associated with the input features and thus adaptively vary corresponding to the data varying across the training phase and the inferring phase.

V. EXPERIMENTS

In this section, we report details of our experimental results, including experimental datasets, settings and implementations, and analyses of experimental results of two pansharpening tasks (i.e., standard HS pansharpening and ARHS pansharpening).

A. Experimental Datasets

The performances of our four PNOs were validated on three HS datasets, which contain a real dataset (i.e., the Merced

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

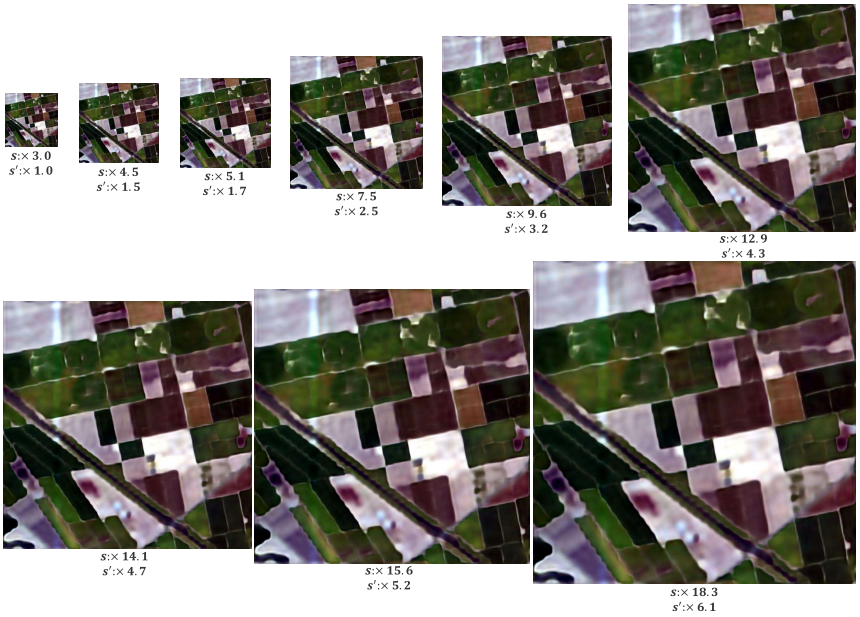


Fig. 9. Visualization of ARHS pansharpening by using eGalerkin-PNO on reduced-resolution Merced dataset.

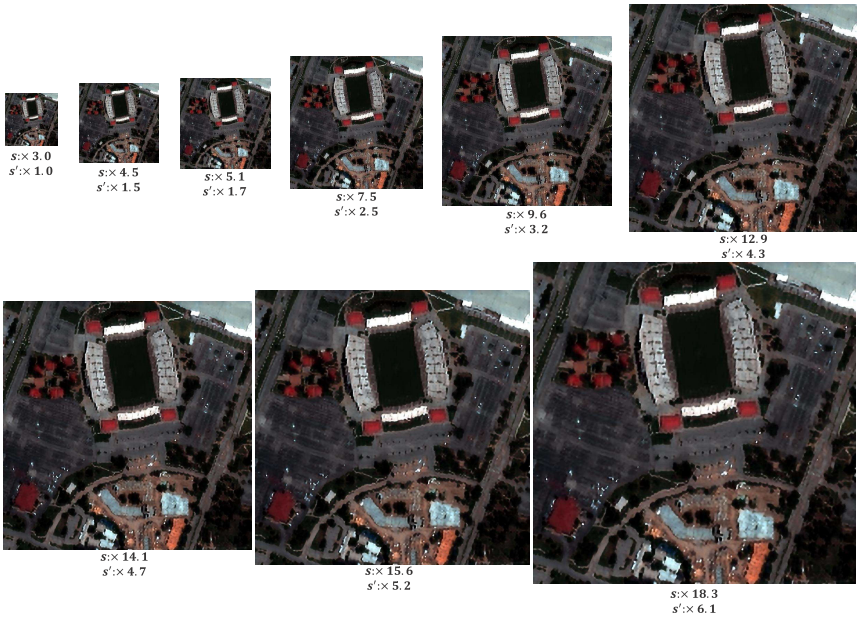


Fig. 10. Visualization of ARHS pansharpening by using eFourier-PNO on reduced-resolution Houston dataset.

dataset) and two simulated datasets (i.e., the Houston dataset and the Washington dataset) [21], [35], [45]. The former is with the PAN observation, while the latter lacks such data, which means the PAN data need to be additionally synthesized. In experiments, some low quality bands are removed from original HS data. Properties of the datasets we used for evaluation were listed in Table I.

B. Experimental Settings

The description of our experimental settings, which include compared pansharpening methods, experimental types, assessment strategies, and experimental configuration.

To ensure a relatively fair comparison, we compared our four PNOs, i.e., Fourier-PNO, Galerkin-PNO, eFourier-PNO, and eGalerkin-PNO, with three categories of algorithms. They are classical model-driven methods, deep-learning methods designed for standard HS pansharpening and methods designed for ARHS pansharpening. Model-driven methods include Gram–Schmidt (GS) [64], principal component analysis (PCA) [65], modulation transfer function-matched generalized Laplacian pyramid with high-pass modulation (MTF-GLP-HPM) [66], and HS superresolution (HySure) [67], while the standard pansharpening methods consist of PanNet [68], HyperPNN [21], HS spectral-fidelity network (HSpeNet) [22], and dual-attention residual network (DARN)

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

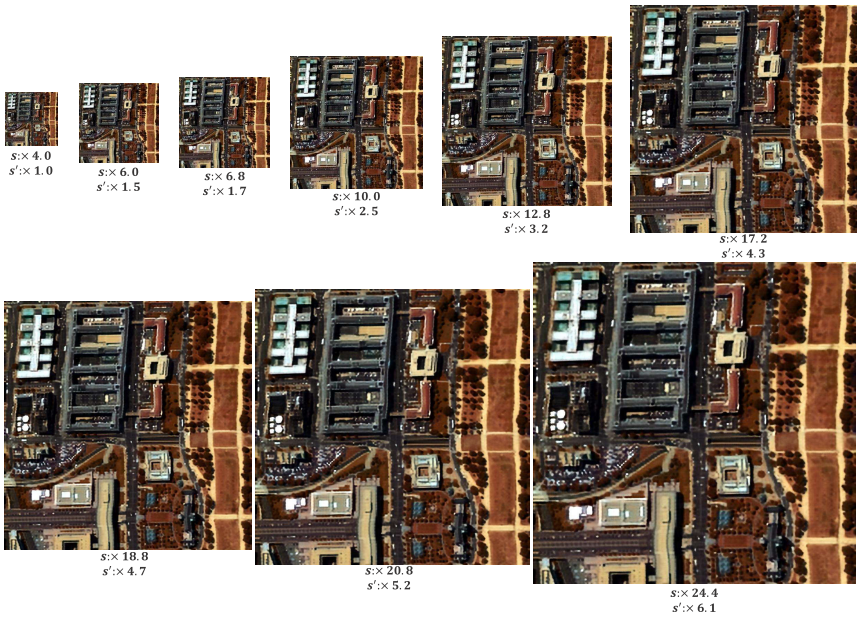


Fig. 11. Visualization of ARHS pansharpening by using eGalerkin-PNO on reduced-resolution Washington dataset.

[23]. Note that these methods were originally designed for standard pansharpening tasks and thus cannot directly fulfill ARHS pansharpening. To address this limitation, we incorporated the interpolation into these methods, which were renamed as GS-Interp, PCA-Interp, MTF-Interp, HySure-Interp, PanNet-Interp, HyperPNN-Interp, HSpeNet-Interp, and DARN-Interp, respectively. To further explore the superiority of our methods, we choose two existing ARHS pansharpening methods, i.e., ARHS-CNN [35] and VSPC-CNN [45]. Therefore, four PNOs were compared with ten existing pansharpening algorithms, i.e., totally 14 methods are performed in comparison.

We conducted two types of pansharpening experiments. One is ARHS pansharpening designed to validate the ability of all the aforementioned methods to reconstruct HS data with any target resolutions, while the other is standard HS pansharpening employed to display their performances in enhancing the spatial resolution of the HS data to that of the associated PAN data. Both of them were assessed on the RR data (i.e., the synthesized data with degrading scale s_t) and the FR data (i.e., the observed data) [35], [44], [45]. In the RR assessment, we employed four quantitative indicators, i.e., cross correlation (CC), spectral angle mapper (SAM), root-mean-square error (RMSE), and peak signal-to-noise ratio (PSNR), to comprehensively calculate the similarity between the pansharpened HS data and the ground-truth (GT, i.e., the observed HS data) [35], [43], [45]. Besides, qualitative visualization pansharpening experiments were also conducted for an intuitive comparison. In the FR assessment, the quality with on reference (QNR), the spatial distortion index D_s and the spectral distortion index D_λ were used for quantitatively evaluate the spectral–spatial fidelity between the reconstructed HS data and the observed HS and PAN data [69], and region of interest (RoI) were shown for qualitatively validation.

All the experiments were conducted in a Linux environment based on Ubuntu 22.04.5 Long-Term Support. Model-driven methods mentioned above were implemented with the pansharpening toolbox provided by the publication [2], and run on a Intel Core i7-10700F CPU. Deep learning methods, i.e., PanNet, HyperPNN, HSpeNet, DARN, ARHS-CNN, VSPC-CNN, Fourier-PNO, Galerkin-PNO, eFourier-PNO, and eGalerkin-PNO, were built by PyTorch version 1.11 and an NVIDIA GeForce GTX 3060 GPU to accelerate the computation and ensure compatibility with the necessary libraries and tools for the tasks. To conduct a relatively convincing comparison, they were set up in accordance with the configurations described in the associated literatures. After 300 epochs learning with the AdamW optimizer [70], [71] setting with weight decay 10^{-4}, beta $[0.9, 0.999]$, and epsilon 10^{-8}, the trained model were applied in inferring phase.

We achieved supervised-learning by training models on the reduced-resolution data, and thus the observed data are needed to be degenerated before we start the optimization of the learning-based pansharpening methods. Regarding the synthesis procedure of the reduced-resolution data, we followed the scale-varied degradation instruction for ARHS pansharpening [35], and the scale-fixed degeneration protocol for standard HS pansharpening [43], [44].

C. Implementation Details

Here, we mainly explain operation details and hyperparameter settings for our PNO methods. For prepansharpening, DARN was employed as Ψ in our four PNOs, and its most hyperparameters are consistent with the descriptions in [23], except that the encoding spectral dimension (i.e., the feature dimension of a) was empirically set to 64. For inverse discretization $\mathcal{T}^{-1}$, bilinear operation was used as the baseline interpolation and s was broadcasted into a tensor with

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

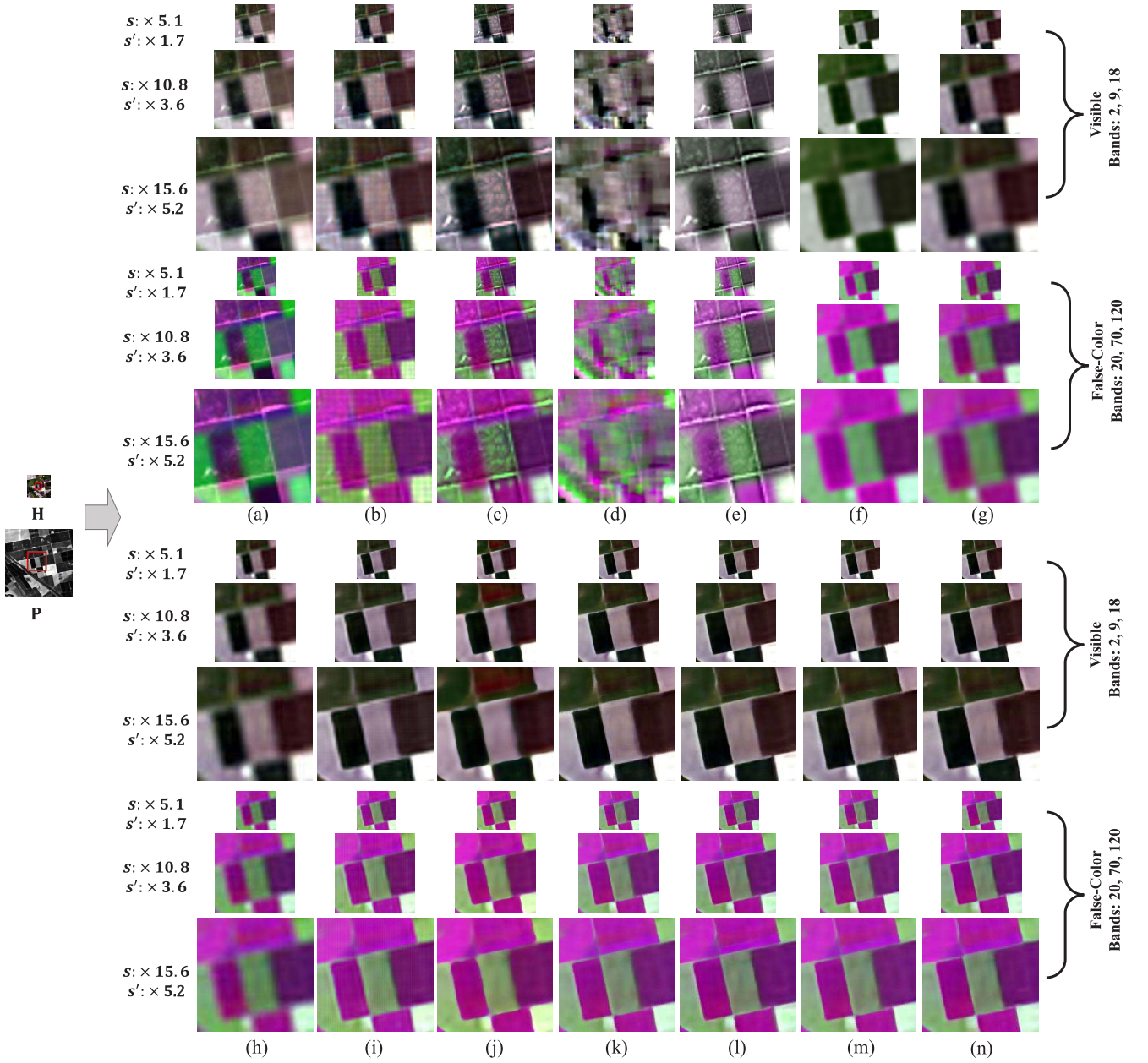


Fig. 12. Visualization of ARHS pansharpening results on full-resolution Merced dataset. Red boxes covering a spatial scope of 60 × 60 LRHS and a spatial scope of 180 × 180 PAN indicate the crop area for demonstration. (a) GS-Interp, (b) PCA-Interp, (c) MTF-Interp, (d) HySure-Interp, (e) PanNet-Interp, (f) HyperPNN-Interp, (g) HSpeNet-Interp, (h) DARN-Interp, (i) ARHS-CNN, (j) VSPC-CNN, (k) Fourier-PNO, (l) Galerkin-PNO, (m) eFourier-PNO, and (n) eGalerkin-PNO.

the spatial shape of $a^{\uparrow}$. As for NO subnetwork $\mathcal{G}$, lifting $\mathcal{P}$ and projection $\mathcal{Q}$ were implemented by the convolution operation, and the dimensions of all the hidden features d_{H_i} were uniformly set to 256. In kernel integrals $\mathcal{K}^{\alpha_{\mathrm{I}}}$ and $\mathcal{K}^{\alpha_{\mathrm{II}}}$, we set the number of iterative layers I to 4, and used 2-D fast Fourier transform to obtain frequency-domain feature. The Fourier modes, i.e., the kernel size of $\boldsymbol{\theta}_i^F$, were empirically set to 10 × 10. Dynamic branches φ were implemented by the convolution. To conduct a stable training, we scaled dynamic Green's functions $\boldsymbol{\theta}^{eF}$ down with a factor $d_{H_i} \times d_{H_i}$. In $\mathcal{K}^{\beta_{\mathrm{I}}}$ and $\mathcal{K}^{\beta_{\mathrm{II}}}$, I was set to 2, and the layer normalization was realized by the column-wise normalization to obtain features consisting of normalized learnable bases. Note that all the activation functions are leaky rectified linear activations with 0.2 negative slope, and all the convolutions are point-wise to keep discretization invariance of the overall model $\mathcal{U}$.

D. ARHS Pansharpening Experiments

To evaluate the results quality of pansharpening across varying spatial resolutions, ARHS pansharpening experiments employed both reduced-resolution and full-resolution experimental assessments.

1) Reduced-Resolution Assessment: Our four PNOs were first compared with ten aforementioned methods by reduced-resolution assessment. In experiments, the intermediate scale s' was set to be 1.7, 2.5, 3.6, and 4.3 to facilitate quantitative

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

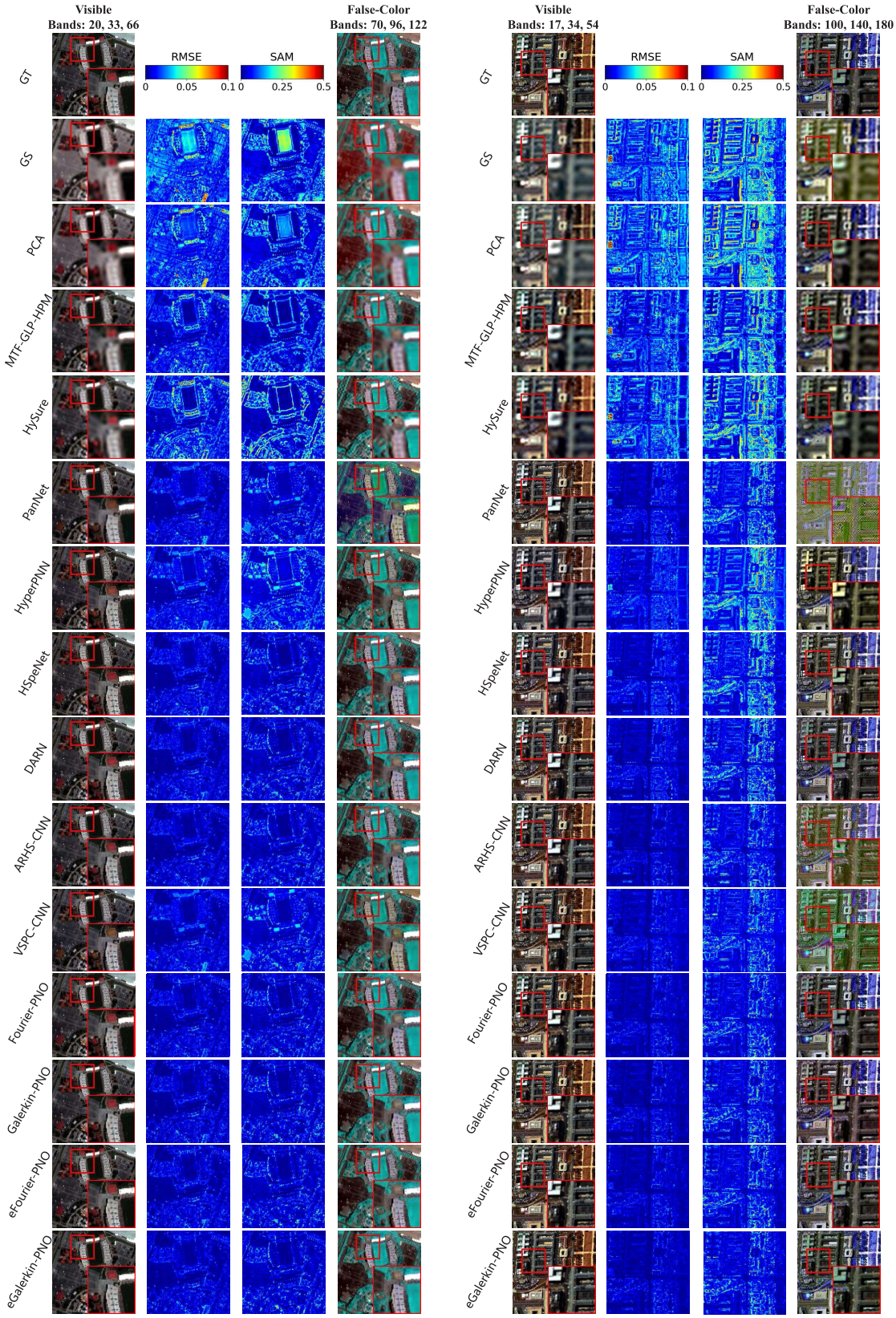


Fig. 13. Visualization of standard HS pansharpening results on reduced-resolution datasets. The HS scenes in the first and the fourth columns of each dataset experiment are displayed in visible and false-color bands, respectively. While the scenes in the second and the third columns are shown in the error figures of SAM and RMSE indicators, respectively. Red boxes covering a spatial scope of 50 × 50 HS image indicate the locally amplified region for better perception. (a) Houston dataset and (b) Washington dataset.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
QUALITY METRICS OF STANDARD HS PANSHARPENING ON THE MERCED DATASET

Methods	CC(↑)	SAM(↓)	RMSE(↓)	PSNR(↑)
GS	0.6933	7.0062	0.0131	34.9059
PCA	0.3525	16.6388	0.0312	30.1157
MTF-GLP-HPM	0.8442	5.2984	0.0080	37.6561
HySure	0.5979	15.4046	0.0308	30.2294
PanNet	0.8944	5.2422	0.0180	39.2616
HyperPNN	0.9438	4.3540	0.0109	41.9893
HSpeNet	0.9568	3.8077	0.0062	42.4437
DARN	0.9575	3.7739	0.0075	43.1436
ARHS-CNN	0.9665	3.7024	0.0070	44.1010
VSPC-CNN	0.9693	3.0059	0.0058	44.6946
Fourier-PNO	0.9703	2.9725	0.0057	44.8063
Galerkin-PNO	0.9714	2.9483	0.0056	44.9983
eFourier-PNO	0.9859	2.4626	0.0051	45.4229
eGalerkin-PNO	**0.9873**	**2.4022**	**0.0049**	**45.6051**

TABLE III
QUALITY METRICS OF STANDARD HS PANSHARPENING ON THE HOUSTON DATASET

Methods	CC(↑)	SAM(↓)	RMSE(↓)	PSNR(↑)
GS	0.9050	4.4062	0.0216	33.3112
PCA	0.9363	3.6768	0.0132	37.6209
MTF-GLP-HPM	0.9151	4.2494	0.0175	35.1523
HySure	0.9208	4.1931	0.0169	35.4182
PanNet	0.9556	3.4347	0.0120	38.4030
HyperPNN	0.9611	3.1447	0.0110	39.1893
HSpeNet	0.9634	3.0224	0.0109	39.2779
DARN	0.9674	2.6621	0.0109	39.2782
ARHS-CNN	0.9707	2.5412	0.0101	39.8900
VSPC-CNN	0.9691	2.5717	0.0103	39.7188
Fourier-PNO	0.9709	2.5353	0.0099	40.0876
Galerkin-PNO	0.9711	2.4874	0.0099	40.1163
eFourier-PNO	0.9786	**2.2527**	**0.0091**	40.4729
eGalerkin-PNO	**0.9789**	2.2554	0.0092	**40.5633**

TABLE IV
QUALITY METRICS OF STANDARD HS PANSHARPENING ON THE WASHINGTON DATASET

Methods	CC(↑)	SAM(↓)	RMSE(↓)	PSNR(↑)
GS	0.8729	7.2183	0.0191	34.3587
PCA	0.8218	7.7761	0.0206	33.7200
MTF-GLP-HPM	0.9044	5.8501	0.0130	37.7451
HySure	0.8960	6.1144	0.0141	37.0182
PanNet	0.9210	5.5865	0.0129	37.7862
HyperPNN	0.9305	4.8491	0.0113	38.9487
HSpeNet	0.9333	4.6461	0.0104	39.6292
DARN	0.9500	4.1533	0.0101	39.9347
ARHS-CNN	0.9531	4.0536	0.0095	40.4676
VSPC-CNN	0.9602	3.5518	0.0083	41.6268
Fourier-PNO	0.9637	3.4400	0.0082	41.7660
Galerkin-PNO	0.9673	3.3722	0.0080	41.9195
eFourier-PNO	0.9737	3.2033	**0.0074**	42.0389
eGalerkin-PNO	**0.9752**	**3.0787**	0.0076	**42.2626**

TABLE V
QUALITY METRICS OF FULL-RESOLUTION EXPERIMENT ON THE MERCED DATASET

Methods	$D_\lambda(\downarrow)$	$D_s(\downarrow)$	QNR(↑)
GS	0.0958	0.1289	0.7877
PCA	0.0701	0.4909	0.4734
MTF-GLP-HPM	0.0745	0.0875	0.8445
HySure	0.0705	0.1244	0.8139
PanNet	0.1730	0.0990	0.7452
HyperPNN	0.0758	0.0524	0.9017
HSpeNet	0.0205	0.0596	0.9281
DARN	0.0154	0.0583	0.9272
ARHS-CNN	0.0194	0.0548	0.9269
VSPC-CNN	0.0403	0.0475	0.9141
Fourier-PNO	0.0229	0.0534	0.9188
Galerkin-PNO	0.0164	0.0257	0.9311
eFourier-PNO	0.0050	0.0244	0.9694
eGalerkin-PNO	**0.0020**	**0.0236**	**0.9744**

and visual comparisons across three datasets, i.e., the ARHS pansharpening scale s was set to be 5.1, 7.5, 10.8, and 12.9 for Merced and Houston datasets, and 6.8, 10.0, 14.4, and 17.2 for the Washington dataset. As for reduced-resolution assessment, observed LRHS images served as GTs, and input LRHS images were the degenerated version of GTs and the associated observed PANs. Such a setup enables the use of quantitative metrics to assess the pansharpening results, as GTs are available for comparison. As shown in Figs. 6–8, our four PNOs consistently outperformed other pansharpening methods across all scales s' on three datasets, especially eGalerkin-PNO. Furthermore, ARHS pansharpening methods exhibited significantly better performance compared to interpolation-based approaches, especially with larger ARHS pansharpening scale s. Then, for a visual evaluation, Figs. 9–11 show the visual results of eGalerkin-PNO and eFourier-PNO. Note that in these three figures, we uniformly set the scale $s' = 1.0, 1.5, 1.7, 2.5, 3.2, 4.3, 4.7, 5.2, 6.1$, i.e., the ARHS pansharpening scale s is set to be 3.0, 4.5, 5.1, 7.5, 9.6, 12.9, 14.1, 15.6, and 18.3 for Merced and Houston datasets, and 4.0, 6.0, 6.8, 10.0, 12.8, 17.2, 18.8, 20.8, and 24.4 for the Washington dataset. Notably, the pansharpening results of both eFourier-PNO and eGalerkin-PNO maintained high spectral–spatial fidelity even as the scale factor s rising.

2) Full-Resolution Assessment: Then, we evaluate the ARHS pansharpening performance of our two enhanced methods (i.e., eFourier-PNO and eGalerkin-PNO) by full-resolution assessment. The experiments can be only performed on the real dataset, i.e., the Merced dataset, due to the necessity of real observed PAN images in the full-resolution indicator D_s. Specifically, the intermediate scale s' was set to be 1.7, 3.6, and 5.2 to fulfill ARHS pansharpening visual comparison, i.e., the corresponding ARHS pansharpening scale s was set to be 5.1, 10.8, and 15.6 on observed Merced data. Fig. 12 shows visualization results in three visible bands and the bands out of the visible range (i.e., false-color bands). It can be observed that the visual results of interpolated-based methods (e.g., MTF-Interp, HySure-Interp, and PanNet-Interp) show obvious spatial distortion, since the interpolation for ARHS pansharpening fails to introduce meaningful textures. Notably, the pansharpened results of model-driven methods exhibit more unnatural details than deep learning-based methods, due to the mismatching between predefined parameters in model-driven methods (e.g., MTF-GLP-HPM and HySure) and data characteristics, whereas deep learning-based methods is able to optimize learnable parameters to adapt to various datasets.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

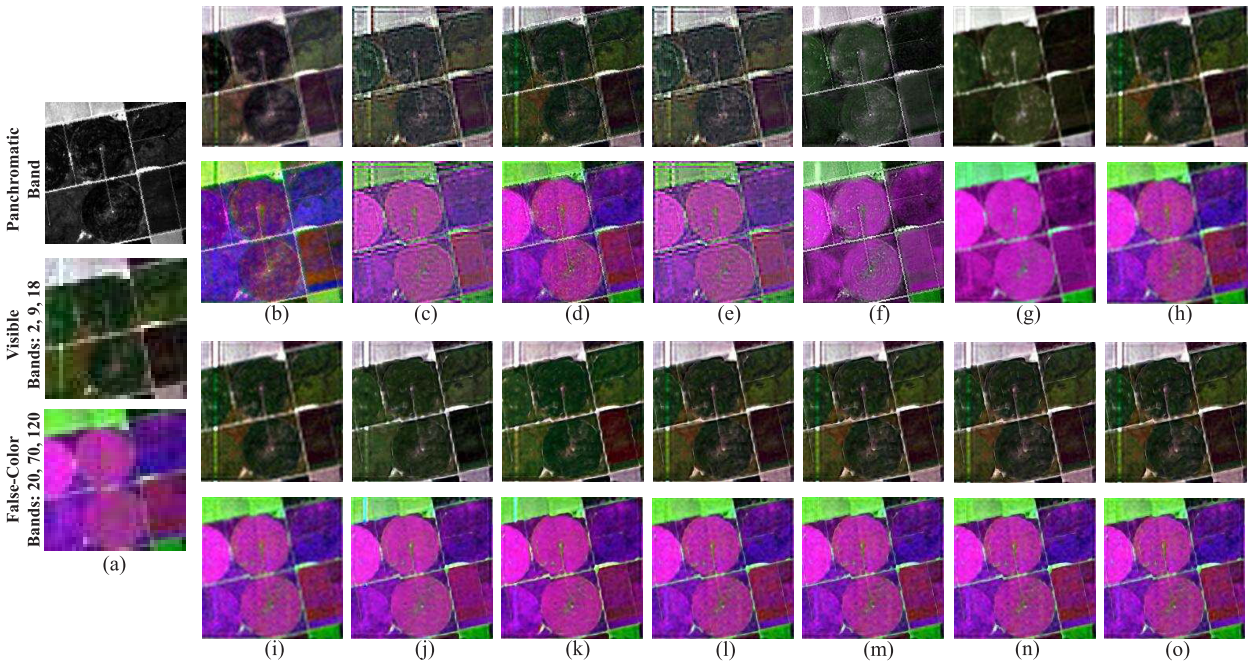


Fig. 14. Visualization of standard HS pansharpening on real Merced dataset. (a) Observed data, (b) GS, (c) PCA, (d) MTF-GLP-HPM, (e) HySure, (f) PanNet, (g) HyperPNN, (h) HSpeNet, (i) DARN, (j) ARHS-CNN, (k) VSPC-CNN, (l) Fourier-PNO, (m) Galerkin-PNO, (n) eFourier-PNO, and (o) eGalerkin-PNO.

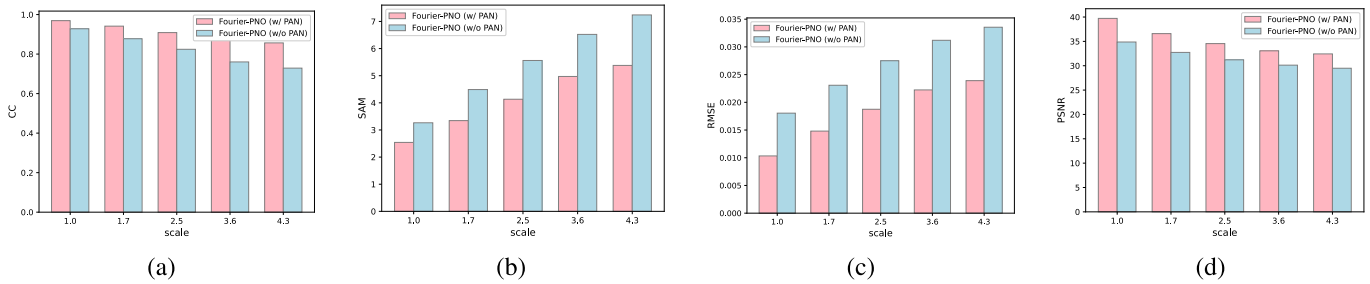


Fig. 15. Ablation experimental results of the proposed Fourier-PNO under arbitrary scales s on Houston dataset, where "w/ PAN" and "w/o PAN" represent the reconstruction process with and without the PAN data, respectively. (a) CC, (b) SAM, (c) RMSE, and (d) PSNR.

Besides, our four PNOs (i.e., Fourier-PNO, Galerkin-PNO, eFourier-PNO, and eGalerkin-PNO) consistently demonstrated the top four visual result quality. In contrast, the results of other methods exhibited varying spectral or spatial distortion degrees, particularly those generated by model-driven methods and standard pansharpening methods.

E. Standard HS Pansharpening Experiments

To further validate the robustness of our methods, we extended our evaluation to standard HS pansharpening experiments, which comprehensively encompassed both reduced-resolution assessment and full-resolution assessment.

1) Reduced-Resolution Assessment: To ensure consistency with the spatial resolution of the observed LRHS images, we configured the scale s' to 1 in ARHS pansharpening, thereby aligning with the standard pansharpening. Standard pansharpening experiments were performed on the Merced dataset, the Houston dataset and the Washington dataset. Besides, we also used four quantitative indicators, i.e., CC, SAM, RMSE, and PSNR to comprehensively assess the quality of pansharpened images.

Tables II–IV show that eGalerkin-PNO and eFourier-PNO consistently ranked either first or second in the quantitative evaluation on all datasets. Furthermore, Fig. 13, i.e., visualization of standard HS pansharpening results on the reduced-resolution Houston and Washington datasets, is composed of four types images, i.e., pansharpened images in visible bands and false-color bands and error images of RMSE and SAM indicators. Whether in Fig. 13(a) or in Fig. 13(b), i.e., the results of the Houston dataset and the Washington dataset, respectively, it can be found that our eFourier-PNO and eGalerkin-PNO reconstruct the results most similar to GTs in visible and false-color bands, and showed the smoothest residual images with the minimal error. That is, the effectiveness of our two enhanced PNOs have been validated.

2) Full-Resolution Assessment: We pansharpened the Merced LRHS data to match the PAN data's resolution for full-resolution assessment, setting the scale $s = 1$. To evaluate the performance of different methods, D_λ, D_s, and QNR were employed. As marked in Table V, eGalerkin-PNO and eFourier-PNO significantly outperformed other methods, especially in SAM indicator, which measures the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

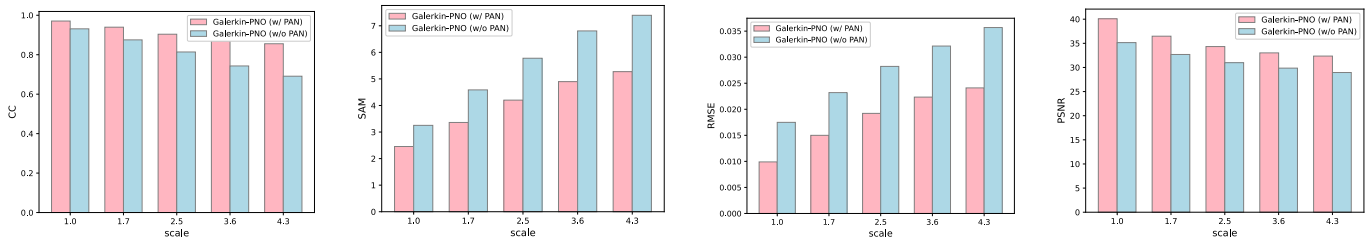


Fig. 16. Ablation experimental results of the proposed Galerkin-PNO under arbitrary scales s on Houston dataset. (a) CC, (b) SAM, (c) RMSE, and (d) PSNR.

spectral-fidelity. Additionally, Fig. 14 shows that the visualization of pansharpened results in both visible and false-color bands demonstrated that the pansharpened images by our methods maintain spectral–spatial fidelity compared to the observed data, further validating the superiority of the proposals.

The above experimental results comprehensively demonstrated the superiorities of our newly designed PNOs, i.e., eGalerkin-PNO, eFourier-PNO, Galerkin-PNO, and Fourier-PNO. Additionally, the effectiveness of the four PNOs on standard HS pansharpening were also verified.

F. Ablation Experiments

We performed the ablation experiments on the reduced-resolution Houston dataset to evaluate the impact of PAN images on the quality of reconstructed HS images across various test scales ($s' = 1.0, 1.7, 2.5, 3.6, 4.3$). The proposed Fourier-PNO and Galerkin-PNO were employed to fulfill the HS pansharpening task including the standard HS pansharpening (i.e., $s' = 1.0$) and ARHS pansharpening (i.e., $s' = 1.7, 2.5, 3.6, 4.3$), and four indicators (i.e., CC, SAM, RMSE and PSNR) are used to comprehensively measure reconstruction performances. It can be observed in Figs. 15 and 16, and the ablation experimental results suggest that PAN data significantly boost both spatial detail preservation and spectral fidelity of reconstructed results, particularly at larger test scales. This implies HS pansharpening offers greater practical significance compared to HSI-SR, as PAN data contain more physically meaningful high-frequency details and authentic textures than LRHS data.

VI. Conclusion and Future Lines

The main challenging problem faced by ARHS pansharpening is to generalize a single pansharpening model trained under training scales to any desired resolutions for the future HS and PAN data. To relieve the problem, we designed four PNOs including Fourier-PNO, Galerkin-PNO, eFourier-PNO, and eGalerkin-PNO. In the design, we introduce continuous spectral–spatial functional spaces and build a PNO scheme comprising prepansharpening encoding, inverse discretization, and functional mapping subnetwork. By use of frequency domain-based kernel integral and Galerkin-based kernel integral, we present Fourier-PNO and Galerkin-PNO. Then, we propose the dynamic Green's function and accordingly develop the innovative eFourier-PNO, and construct PAN-guided test space and then develop the novel eGalerkin-PNO. The pansharpening performances of our PNOs were evaluated on several datasets.

Our PNOs are originally built in continuous spectral–spatial functional spaces. Therefore, they have potentials to be extended for spectral–spatial joint ARHS pansharpening, which fuses LRHS data and PAN data to promote HS images along the spatial dimension and the spectral dimension simultaneously. In the future, we may improve our work in this direction.

References

[1] G. Shaw and D. Manolakis, "Signal processing for hyperspectral image exploitation," *IEEE Signal Process. Mag.*, vol. 19, no. 1, pp. 12–16, Jan. 2002.

[2] L. Loncan et al., "Hyperspectral pansharpening: A review," *IEEE Geosci. Remote Sens. Mag. Replaces Newsletter*, vol. 3, no. 3, pp. 27–46, Sep. 2015.

[3] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral–spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2018.

[4] X. Xu, J. Li, S. Li, and A. Plaza, "Generalized morphological component analysis for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2817–2832, Apr. 2020.

[5] N. M. Nasrabadi, "Hyperspectral target detection : An overview of current and future challenges," *IEEE Signal Process. Mag.*, vol. 31, no. 1, pp. 34–44, Jan. 2014.

[6] D. Marinelli, F. Bovolo, and L. Bruzzone, "A novel change detection method for multitemporal hyperspectral images based on binary hyperspectral change vectors," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4913–4928, Jul. 2019.

[7] X. Wang, Q. Hu, Y. Cheng, and J. Ma, "Hyperspectral image super-resolution meets deep learning: A survey and perspective," *IEEE/CAA J. Autom. Sinica*, vol. 10, no. 8, pp. 1668–1691, Aug. 2023.

[8] J. Li et al., "Deep learning in multimodal remote sensing data fusion: A comprehensive review," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 112, Aug. 2022, Art. no. 102926.

[9] N. Aburaed, M. Q. Alkhatib, S. Marshall, J. Zabalza, and H. A. Ahmad, "A review of spatial enhancement of hyperspectral remote sensing imaging techniques," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 16, pp. 2275–2300, 2023.

[10] B. Aiazzi, S. Baronti, and M. Selva, "Improving component substitution pansharpening through multivariate regression of MS +pan data," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 10, pp. 3230–3239, Oct. 2007.

[11] V. P. Shah, N. H. Younan, and R. L. King, "An efficient pan-sharpening method via a combined adaptive PCA approach and contourlets," *IEEE Trans. Geosci. Remote Sens.*, vol. 46, no. 5, pp. 1323–1335, May 2008.

[12] T.-M. Tu, P. S. Huang, C.-L. Hung, and C.-P. Chang, "A fast intensity-hue-saturation fusion technique with spectral adjustment for IKONOS imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 1, no. 4, pp. 309–312, Oct. 2004.

[13] J. Lee and C. Lee, "Fast and efficient panchromatic sharpening," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 1, pp. 155–163, Jan. 2010.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

[14] Q. Wei, N. Dobigeon, and J.-Y. Tourneret, "Bayesian fusion of multi-band images," *IEEE J. Sel. Topics Signal Process.*, vol. 9, no. 6, pp. 1117–1127, Sep. 2015.

[15] M. R. Vicinanza, R. Restaino, G. Vivone, M. D. Mura, and J. Chanussot, "A pansharpening method based on the sparse representation of injected details," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 1, pp. 180–184, Jan. 2015.

[16] F. D. Javan, F. Samadzadegan, S. Mehravar, A. Toosi, R. Khatami, and A. Stein, "A review of image fusion techniques for pan-sharpening of high-resolution satellite imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 171, pp. 101–117, Jan. 2021.

[17] N. Yokoya, T. Yairi, and A. Iwasaki, "Coupled nonnegative matrix factorization unmixing for hyperspectral and multispectral data fusion," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 2, pp. 528–537, Feb. 2011.

[18] B. Huang, H. Song, H. Cui, J. Peng, and Z. Xu, "Spatial and spectral image fusion using sparse matrix factorization," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 3, pp. 1693–1704, Mar. 2014.

[19] M. Moeller, T. Wittman, and A. L. Bertozzi, "A variational approach to hyperspectral image fusion," *Proc. SPIE*, vol. 7334, pp. 502–511, Apr. 2009.

[20] G. Masi, D. Cozzolino, L. Verdoliva, and G. Scarpa, "Pansharpening by convolutional neural networks," *Remote Sens.*, vol. 8, no. 7, p. 594, 2016.

[21] L. He, J. Zhu, J. Li, A. Plaza, J. Chanussot, and B. Li, "HyperPNN: Hyperspectral pansharpening via spectrally predictive convolutional neural networks," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 8, pp. 3092–3100, Aug. 2019.

[22] L. He, J. Zhu, J. Li, D. Meng, J. Chanussot, and A. Plaza, "Spectral-fidelity convolutional neural networks for hyperspectral pansharpening," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 5898–5914, 2020.

[23] W. Xie, J. Lei, Y. Cui, Y. Li, and Q. Du, "Hyperspectral pansharpening with deep priors," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 5, pp. 1529–1543, May 2020.

[24] Y. Zheng, J. Li, Y. Li, J. Guo, X. Wu, and J. Chanussot, "Hyperspectral pansharpening using deep prior and dual attention residual network," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8059–8076, Nov. 2020.

[25] W. G. C. Bandara, J. M. J. Valanarasu, and V. M. Patel, "Hyperspectral pansharpening based on improved deep image prior and residual reconstruction," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5520816.

[26] W. Dong, Y. Yang, J. Qu, Y. Li, Y. Yang, and X. Jia, "Feature pyramid fusion network for hyperspectral pansharpening," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 1555–1567, Jan. 2025.

[27] W. G. C. Bandara and V. M. Patel, "HyperTransformer: A textural and spectral feature fusion transformer for pansharpening," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Aug. 2022, pp. 1767–1777.

[28] J. Qu, W. Dong, Y. Li, S. Hou, and Q. Du, "An interpretable unsupervised unrolling network for hyperspectral pansharpening," *IEEE Trans. Cybern.*, vol. 53, no. 12, pp. 7943–7956, Dec. 2023.

[29] J. Qu et al., "Progressive multi-iteration registration-fusion co-optimization network for unregistered hyperspectral image super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5519814.

[30] M. Zhang et al., "Spatial–spectral aggregation transformer with diffusion prior for hyperspectral image super-resolution," *IEEE Trans. Circuits Syst. Video Technol.*, early access, Nov. 29, 2024, doi: 10.1109/TCSVT.2024.3508844.

[31] J. Cui, W. Dong, J. Qu, X. Wu, S. Xiao, and Y. Li, "CFMDM: Coarse-to-fine meta-diffusion model for scale-arbitrary hyperspectral super-resolution," *IEEE Geosci. Remote Sens. Lett.*, vol. 21, pp. 1–5, 2024.

[32] W. Dong, S. Liu, S. Xiao, J. Qu, and Y. Li, "ISPDiff: Interpretable scale-propelled diffusion model for hyperspectral image super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5519614.

[33] J. Liu, Z. Wu, L. Xiao, and X.-J. Wu, "Model inspired autoencoder for unsupervised hyperspectral image super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522412.

[34] J. Liu, Z. Wu, and L. Xiao, "A spectral diffusion prior for unsupervised hyperspectral image super-resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5528613.

[35] L. He, J. Zhu, J. Li, A. Plaza, J. Chanussot, and Z. Yu, "CNN-based hyperspectral pansharpening with arbitrary resolution," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518821.

[36] F. Laporterie-Déjean, H. de Boissezon, G. Flouzat, and M.-J. Lefèvre-Fonollosa, "Thematic and statistical evaluations of five panchromatic/multispectral fusion methods on simulated PLEIADES-HR images," *Inf. Fusion*, vol. 6, no. 3, pp. 193–212, 2005.

[37] P. Wang, L. Wang, Y. Wu, and H. Leung, "Utilizing pansharpening technique to produce sub-pixel resolution thematic map from coarse remote sensing image," *Remote Sens.*, vol. 10, no. 6, p. 884, Jun. 2018.

[38] P. Sirguey, R. Mathieu, Y. Arnaud, M. M. Khan, and J. Chanussot, "Improving MODIS spatial resolution for snow mapping using wavelet fusion and ARSIS concept," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 1, pp. 78–82, Jan. 2008.

[39] J. B. Adams and A. R. Gillespie, *Remote Sensing of Landscapes With Spectral Images: A Physical Modeling Approach*. Cambridge, U.K.: Cambridge Univ. Press, Nov. 2018.

[40] Y. Qu, H. Qi, B. Ayhan, C. Kwan, and R. Kidd, "DOES multispectral/hyperspectral pansharpening improve the performance of anomaly detection?" in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2017, pp. 6130–6133.

[41] A. Mohammadzadeh, A. Tavakoli, and M. J. Valadan Zoej, "Road extraction based on fuzzy logic and mathematical morphology from pan-sharpened IKONOS images," *Photogram. Rec.*, vol. 21, no. 113, pp. 44–60, Mar. 2006.

[42] Y. Xu, S. E. Smith, S. Grunwald, A. Abd-Elrahman, and S. P. Wani, "Incorporation of satellite remote sensing pan-sharpened imagery into digital soil prediction and mapping models to characterize soil property variability in small agricultural fields," *ISPRS J. Photogramm. Remote Sens.*, vol. 123, pp. 1–19, Jan. 2017.

[43] L. Wald, T. Ranchin, and M. Mangolini, "Fusion of satellite images of different spatial resolutions: Assessing the quality of resulting images," *Photogramm. Eng. Remote Sens.*, vol. 63, no. 6, pp. 691–699, 1997.

[44] G. Vivone, M. Dalla Mura, A. Garzelli, and F. Pacifici, "A benchmarking protocol for pansharpening: Dataset, preprocessing, and quality assessment," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 14, pp. 6102–6118, 2021.

[45] L. He, J. Xie, J. Li, A. Plaza, J. Chanussot, and J. Zhu, "Variable subpixel convolution based arbitrary-resolution hyperspectral pansharpening," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5534519.

[46] Z. Li et al., "Neural operator: Graph kernel network for partial differential equations," 2020, *arXiv:2003.03485*.

[47] Z. Li et al., "Fourier neural operator for parametric partial differential equations," 2020, *arXiv:2010.08895*.

[48] Z. Li et al., "Multipole graph neural operator for parametric partial differential equations," in *Proc. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 33, Apr. 2020, pp. 6755–6766.

[49] N. Kovachki et al., "Neural operator: Learning maps between function spaces with applications to PDEs," *J. Mach. Learn. Res.*, vol. 24, no. 89, pp. 1–97, 2023.

[50] Y. Yang, A. F. Gao, K. Azizzadenesheli, R. W. Clayton, and Z. E. Ross, "Rapid seismic waveform modeling and inversion with neural operators," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5906712.

[51] B. Li, H. Wang, S. Feng, X. Yang, and Y. Lin, "Solving seismic wave equations on variable velocity models with Fourier neural operator," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4508618.

[52] Z. Peng, B. Yang, Y. Xu, F. Wang, L. Liu, and Y. Zhang, "Rapid surrogate modeling of electromagnetic data in frequency domain using neural operator," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 2007912.

[53] Y. Chen, S. Liu, and X. Wang, "Learning continuous image representation with local implicit image function," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jul. 2021, pp. 8628–8638.

[54] J. Lee and K. H. Jin, "Local texture estimator for implicit representation function," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 1919–1928.

[55] J.-E. Yao, L.-Y. Tsao, Y.-C. Lo, R. Tseng, C.-C. Chang, and C.-Y. Lee, "Local implicit normalizing flow for arbitrary-scale image super-resolution," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 1776–1785.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.

[56] S. S. Bayin, "Green's functions," in *Mathematical Methods in Science and Engineering*. Hoboken, NJ, USA: Wiley, Mar. 2006, ch. 1, pp. 567–632.

[57] M. Barkat, "Representation of signals," in *Signal Detection and Estimation*. Norwood, MA, USA: Artech House, Jan. 2005, ch. 8, pp. 449–502.

[58] A. D. Polyanin, "Construction of the green's functions," in *Handbook of Linear Partial Differential Equations for Engineers and Scientists*. London, U.K.: Chapman & Hall, 2001, ch. 9, pp. 32–35.

[59] A. V. Oppenheim and G. C. Verghese, *Signals, Systems & Inference*. London, U.K.: Pearson, Jul. 2017.

[60] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[61] S. Cao, "Choose a transformer: Fourier or Galerkin," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 24924–24940.

[62] A. Ern and J.-L. Guermond, *Theory and Practice of Finite Elements*, vol. 159. Cham, Switzerland: Springer, Jun. 2004.

[63] Q. Yang et al., "Fourier neural operators for arbitrary resolution climate data downscaling," 2023, *arXiv:2305.14452*.

[64] C. A. Laben and B. V. Brower, "Process for enhancing the spatial resolution of multispectral imagery using pan-sharpening," U.S. Patent 6 011 875, Jan. 4, 2000.

[65] P. Kwarteng and A. Chavez, "Extracting spectral contrast in Landsat thematic mapper image data using selective principal component analysis," *Photogramm. Eng. Remote Sens.*, vol. 55, no. 1, pp. 339–348, 1989.

[66] G. Vivone, R. Restaino, M. D. Mura, G. Licciardi, and J. Chanussot, "Contrast and error-based fusion schemes for multispectral image pansharpening," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 5, pp. 930–934, May 2014.

[67] M. Sim oes, J. Bioucas-Dias, L. B. Almeida, and J. Chanussot, "Hyperspectral image superresolution: An edge-preserving convex formulation," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2014, pp. 4166–4170.

[68] J. Yang, X. Fu, Y. Hu, Y. Huang, X. Ding, and J. Paisley, "PanNet: A deep network architecture for pan-sharpening," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 5449–5457.

[69] L. Alparone, B. Aiazzi, S. Baronti, A. Garzelli, F. Nencini, and M. Selva, "Multispectral and panchromatic data fusion assessment without reference," *Photogramm. Eng. Remote Sens.*, vol. 74, no. 2, pp. 193–200, Feb. 2008.

[70] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2018, pp. 1–8.

[71] A. Fawzi et al., "Discovering faster matrix multiplication algorithms with reinforcement learning," *Nature*, vol. 610, no. 7930, pp. 47–53, Oct. 2022.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 20:51:06 UTC from IEEE Xplore. Restrictions apply.